

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

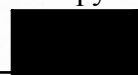
Дисциплина:

«Технологии и методы программирования»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

Выполнили:

Жук Анастасия Валерьевна,
студентка группы N3348



(подпись)

Проверил:

Ищенко Алексей Петрович

(отметка о выполнении)

(подпись)

Санкт-Петербург

2025 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
ХОД РАБОТЫ.....	4
А. Локальная операционная система.....	4
Техническое задание.....	4
Описание продукта, его функционал.....	6
Инструкция по применению.....	8
Код на языке программирования Python.....	12
Листинг 1 – Код для сборки exe файлов build.py.....	12
Листинг 2 – Код программы инсталлятора sys_doc.py для sys_doc.exe.....	16
Листинг 3 – Код программы-защитника secur.py для secur.exe.....	28
Примеры работы программы.....	35
Б. Локальная сеть.....	41
Техническое задание.....	41
Описание продукта, его функционал.....	42
Инструкция по применению.....	44
Код на языке программирования Python.....	49
Листинг 4 – Код сервера server.py.....	49
Листинг 5 – Код анализатора analyzer.py.....	68
Примеры работы программы.....	74
ЗАКЛЮЧЕНИЕ.....	79

ВВЕДЕНИЕ

Цель работы – разработка комплексной системы, демонстрирующей два подхода к работе с информацией: защиту локальных системных данных с использованием криптографии и удаленный сбор технической информации через веб-интерфейсы.

Для достижения поставленной цели необходимо решить следующие **задачи**:

- создать систему шифрования и электронной подписи для защиты системной информации;
- разработать графический инсталлятор, интегрирующий защитные механизмы в ОС Windows;
- реализовать программу контроля доступа с проверкой цифровой подписи;
- создать веб-сервер для скрытого сбора технических данных об устройствах пользователей;
- разработать инструмент анализа и визуализации собранной статистики;
- обеспечить работу системы через сетевое хранилище данных.

ХОД РАБОТЫ

А. Локальная операционная система

Техническое задание

1. Создать текстовый документ (sys.tat), в котором будет содержаться «Системная информация».
2. Написать программу-инсталлятор sys_doc.exe для этого документа, которая под видом установки обновления (с отображением строки прогресса обновления) к какой-нибудь программе (например, Блокнот или Paint):
 - запрашивает у пользователя папку (должен быть вариант использования существующей папки и вариант создания собственной) для копирования «Системной информации»;
 - записывает в папку файл с исполняемым кодом программы secur.exe (аналог требований к template.tbl из лабораторной работы №1), защищающей sys.tat;
 - собирает (возможную) информацию о компьютере, на котором устанавливается программа;
 - кодирует эту информацию и записывает в файл sys.tat;
 - подписывает ее личным ключом пользователя программы и записывает подпись, например, в реестр Windows в раздел HKEY_CURRENT_USER\Software\Фамилия_студента как значение параметра Signature;
 - запускает secur.exe для защиты sys.tat от несанкционированного доступа;
 - прописывает запуск программы secur.exe при выполнении функции Open для sys.tat, чтобы защита срабатывала и после перезагрузки ОС (есть несколько способов такой «привязки»).
3. В саму программу защиты secur.exe включить следующий функционал:

- запрос у пользователя информации об имени раздела реестра с электронной цифровой подписью (фамилией студента);
 - считывание подписи из указанного выше раздела реестра, которая проверяется с помощью открытого ключа пользователя;
 - разрешение или запрет просмотра «Системной информации» в файле sys.tat в зависимости от правильности указания ключа;
4. При неудачной проверке работа защищаемой программы должна прекращаться с выдачей соответствующего сообщения.
5. Собираемая о компьютере информация включает в себя как минимум:
- Имя пользователя,
 - Имя компьютера,
 - Конфигурацию компьютера (память и процессор, как минимум) и версию ОС.

Описание продукта, его функционал

Разработанный программный комплекс представляет собой законченное решение для сбора, шифрования, цифровой подписи и защищенного хранения системной информации компьютера. Система реализует полный цикл защиты данных от момента их сбора до безопасного просмотра, демонстрируя практическое применение современных криптографических методов в реальных условиях.

Архитектура системы включает четыре ключевых компонента, взаимодействующих между собой.

1. **Графический инсталлятор** (Обновление Paint.exe) – основное приложение, которое маскируется под обновление популярного графического редактора, обеспечивая удобный интерфейс для установки всей системы защиты. Эта программа выполняет инициализацию, сбор данных и настройку защитных механизмов.

2. **Программа-защитник** (secur.exe) – специализированное приложение, ответственное за проверку целостности данных и предоставление авторизованного доступа к защищенной информации. Этот компонент реализует механизмы проверки электронной подписи и управления доступом.

3. **Защищенный файл данных** (sys.tat) – зашифрованный контейнер специального формата, содержащий системную информацию в защищенном виде. Файл имеет собственную структуру, включающую заголовок и зашифрованное содержимое в кодировке base64.

4. **Ключевая инфраструктура** – набор криптографических ключей, включая пару RSA-ключей для цифровой подписи и симметричный ключ Fernet для шифрования данных. Особенностью реализации является то, что приватный ключ RSA никогда не сохраняется на диск, оставаясь только в оперативной памяти на время создания подписи.

Функциональные возможности системы охватывают весь процесс работы с защищенными данными:

- *сбор системной информации* – автоматическое получение детальных сведений об операционной системе, процессоре, оперативной памяти, сетевых настройках и пользовательском окружении;

- *двойное шифрование* – применение гибридной схемы: симметричное шифрование Fernet для данных и асимметричное RSA для управления ключами;

- *цифровая подпись* – создание электронной подписи от зашифрованного файла с последующим хранением подписи в системном реестре Windows;

- *контроль доступа* – проверка подлинности данных через сравнение цифровой подписи с использованием публичного ключа;

- *интеграция с ОС* – создание ассоциации файлов .tat с программой-защитником, обеспечивающей автоматический запуск проверки при открытии файлов данного типа.

Технические особенности реализации подчеркивают внимание к безопасности:

- приватный ключ RSA генерируется, используется для создания подписи и немедленно удаляется из памяти, что исключает его компрометацию;

- электронная подпись создается от уже зашифрованного файла, обеспечивая защиту как содержимого, так и целостности контейнера;

- система проверяет наличие всех необходимых компонентов (ключей, файлов данных, записей реестра) перед предоставлением доступа к информации;

- реализована обработка различных сценариев ошибок с сообщениями.

Инструкция по применению

Этап 1: Подготовка и сборка

Перед началом работы необходимо подготовить среду выполнения. Требуется установить интерпретатор Python версии 3.8 или выше, а также дополнительные библиотеки, используемые в проекте. Установка выполняется через менеджер пакетов `pip` одной командой, включающей **pyinstaller** для создания исполняемых файлов, **cryptography** для криптографических операций и **psutil** для сбора системной информации.

После установки зависимостей запускается скрипт сборки **build.py**, который автоматически создает два исполняемых файла: графический инсталлятор и программу-защитник. Процесс сборки отображает подробный лог, включая этапы компиляции, копирования файлов и проверки результатов. По завершении сборки на рабочем столе появляется файл Обновление Paint.exe, готовый к использованию.

Этап 2: Установка системы защиты

Запуск инсталлятора открывает графический интерфейс, стилизованный под официальное обновление Microsoft. Пользователю предлагается выбрать папку для установки — можно указать существующий путь или создать новую директорию.

После выбора папки начинается процесс установки, отображаемый через прогресс-бар и текстовый лог в реальном времени. Система последовательно выполняет следующие операции:

- создание структуры папок,
- копирование программы-защитника,
- настройка ассоциации файлов .tat,
- сбор информации о системе,
- генерация криптографических ключей,
- шифрование данных и сохранение в файл sys.tat,
- создание цифровой подписи и ее запись в реестр,

- безопасное удаление приватного ключа из памяти.

По завершении установки появляется сообщение об успехе.

Этап 3: Проверка установки и конфигурации

После успешной установки в выбранной папке создаются следующие файлы:

- **sys.tat** — основной контейнер с зашифрованной системной информацией
- **public_key.pem** — публичный ключ RSA в формате PEM
- **fernet.key** — симметричный ключ для алгоритма Fernet
- **secur.exe** — исполняемый файл программы-защитника

Хочется сразу отметить, что в реальных проектах ключи лучше всего хранить в месте, далеком от обычных пользователей, или там, куда у обычных пользователей нет доступа.

Для верификации настроек рекомендуется проверить запись в системном реестре Windows. В разделе HKEY_CURRENT_USER\Software\Zhuk должны присутствовать три строковых параметра: Signature (электронная подпись в base64), InstallPath (путь к папке установки) и PublicKeyPath (путь к файлу с публичным ключом). Наличие этих записей подтверждает корректность настройки системы.

Этап 4: Работа с защищенной информацией

Система предоставляет два равноценных способа доступа к защищенным данным:

Первый способ использует ассоциацию файлов в операционной системе. После установки файлы с расширением .tat автоматически связываются с программой-защитником. При двойном щелчке по файлу sys.tat запускается secur.exe, который запрашивает фамилию студента (по умолчанию "Zhuk"), проверяет цифровую подпись и при успешной проверке отображает расшифрованную системную информацию.

Второй способ предполагает прямой запуск программы-защитника. Пользователь может открыть `secur.exe` из папки установки, после чего программа автоматически определит расположение файла `sys.tat`, запросит фамилию для доступа к реестру и выполнит всю последовательность проверок.

В обоих случаях процесс включает одинаковые этапы:

1. Запрос идентификационной информации (фамилии студента).
2. Чтение цифровой подписи из соответствующего раздела реестра.
3. Загрузка публичного ключа из файла.
4. Проверка электронной подписи файла `sys.tat`.
5. Поиск ключа шифрования Fernet.
6. Расшифровка и отображение системной информации.

Этап 5: Типовые сценарии использования и отказоустойчивость

Система разработана с учетом различных сценариев использования и включает механизмы обработки исключительных ситуаций.

Штатный режим работы: пользователь с правильной фамилией получает полный доступ к системной информации, которая отображается в структурированном виде с разделами: пользователь и компьютер, операционная система, процессор, оперативная память, сетевые настройки и метаданные.

Попытка несанкционированного доступа: при вводе неверной фамилии система не находит соответствующей записи в реестре и прекращает работу с сообщением «ДОСТУП ЗАПРЕЩЕН!». Этот механизм обеспечивает базовый уровень контроля доступа.

Нарушение целостности данных: если содержимое файла `sys.tat` было изменено после создания подписи, проверка электронной подписи завершится неудачей с соответствующим предупреждением. Система детектирует даже минимальные изменения в защищенном файле.

Потеря компонентов: при отсутствии необходимых файлов (ключа шифрования, публичного ключа или самого файла данных) программа сообщает о конкретной проблеме и предлагает возможные пути решения, например, поиск ключа в альтернативных расположениях.

Безопасность и ограничения

Разработанная система реализует несколько уровней защиты информации. Криптографическая защита обеспечивается гибридной схемой шифрования и цифровой подписи. Управление доступом осуществляется через связку «фамилия студента – запись в реестре». Целостность данных гарантируется проверкой электронной подписи.

Важным аспектом безопасности является подход к хранению ключей: приватный ключ RSA существует только в оперативной памяти в момент создания подписи и никогда не сохраняется на диск, что существенно снижает риск его компрометации.

При использовании системы следует учитывать, что некоторые антивирусные программы могут классифицировать созданные .exe файлы как потенциально опасные из-за их поведения (работа с реестром, шифрование данных). В таких случаях рекомендуется добавить папку с программой в исключения антивируса или использовать виртуальную машину для тестирования.

Система требует прав на запись в выбранную папку установки и доступ к реестру текущего пользователя. При работе в корпоративной среде с ограниченными правами пользователя может потребоваться запуск от имени администратора.

Код на языке программирования Python

Листинг 1 – Код для сборки exe файлов build.py

```
import os
import sys
import subprocess
import shutil

def build_project():
    print("=" * 70)
    print("СБОРКА ПРОЕКТА - ЧАСТЬ А")
    print("=" * 70)

    # Создание папок
    os.makedirs("dist", exist_ok=True)
    os.makedirs("build", exist_ok=True)

    # 1. Сборка secur.exe
    print("\n1. Сборка secur.exe...")
    try:
        result = subprocess.run([
            sys.executable, "-m", "PyInstaller",
            "--onefile",
            "--name=secur",
            "--distpath=./dist",
            "--workpath=./build",
            "--hidden-import=cryptography.hazmat.backends.openssl",
            "--hidden-import=cryptography.hazmat.primitives",
            "--hidden-import=cryptography.hazmat.primitives.asymmetric",
            "secur.py"
        ], capture_output=True, text=True, check=True)

        print(" secur.exe создан")

        # Копируем в текущую папку
        if os.path.exists("dist/secur.exe"):
            shutil.copy("dist/secur.exe", "secur.exe")
            print(" secur.exe скопирован")

    except subprocess.CalledProcessError as e:
        print(f" Ошибка: {e.stderr}")
    except Exception as e:
        print(f" Ошибка: {e}")

    # 2. Сборка графического инсталлятора
    print("\n2. Сборка графического инсталлятора...")
    try:
        # Создаем спес файл для сборки с данными
        espec_content = ""
    except Exception as e:
        print(f" Ошибка: {e}")

# -*- mode: python ; coding: utf-8 -*-
```



```
import sys
from PyInstaller.utils.hooks import collect_data_files
```

```
a = Analysis(
    ['sys_doc.py'],
    pathex=[],
    binaries=[],
    datas=[('secur.exe', '.')],
    hiddenimports=[
        'cryptography.hazmat.backends.openssl',
        'cryptography.hazmat.primitives',
        'cryptography.hazmat.primitives.asymmetric',
        'psutil._psutil_windows',
        'psutil._psutil_posix',
        'tkinter'
    ],
    hookspath=[],
    hooksconfig={},
    runtime_hooks=[],
    excludes=[],
    noarchive=False,
    optimize=0
)
```

```
pyz = PYZ(a.pure)
```

```
exe = EXE(
    pyz,
    a.scripts,
    a.binaries,
    a.datas,
    [],
    name='Обновление Paint',
    debug=False,
    bootloader_ignore_signals=False,
    strip=False,
    upx=True,
    upx_exclude=[],
    runtime_tmpdir=None,
    console=False,
    disable_windowed_traceback=False,
    argv_emulation=False,
    target_arch=None,
    codesign_identity=None,
    entitlements_file=None,
    icon=None
)
"""
```

```
# Сохраняем спес файл
```

```

with open("sys_doc.spec", "w", encoding="utf-8") as f:
    f.write(spec_content)

# Собираем с помощью spec
result = subprocess.run([
    sys.executable, "-m", "PyInstaller",
    "--clean",
    "sys_doc.spec"
], capture_output=True, text=True, check=True)

print(" Обновление Paint.exe создан")

# Копируем на рабочий стол
desktop = os.path.join(os.path.expanduser("~"), "Desktop")
if os.path.exists("dist/Обновление Paint.exe"):
    dest = os.path.join(desktop, "Обновление Paint.exe")
    shutil.copy("dist/Обновление Paint.exe", dest)
    print(f" Скопировано на рабочий стол: {dest}")

except subprocess.CalledProcessError as e:
    print(f" Ошибка сборки GUI: {e.stderr}")
except Exception as e:
    print(f" Ошибка: {e}")

# 3. Проверка наличия файлов
print("\n3. Проверка созданных файлов...")
created_files = []

if os.path.exists("secur.exe"):
    created_files.append("secur.exe")

desktop_file = os.path.join(os.path.expanduser("~"), "Desktop", "Обновление Paint.exe")
if os.path.exists(desktop_file):
    created_files.append("Обновление Paint.exe (на рабочем столе)")

if os.path.exists("dist/Обновление Paint.exe"):
    created_files.append("dist/Обновление Paint.exe")

if created_files:
    print(" Созданы файлы:")
    for file in created_files:
        print(f" • {file}")
else:
    print(" Файлы не созданы!")

# 4. Создание README
print("\n4. Создание инструкции...")
readme_content = """
# Лабораторная работа №3 - Часть А
## Автор: Жук Анастасия

```

ИНСТРУКЦИЯ ПО ЗАПУСКУ

Файлы проекта:

1. Обновление Paint.exe - графический инсталлятор
2. secur.exe - программа просмотра защищенных данных

Шаги установки:

1. Запустите Обновление Paint.exe (на рабочем столе)
2. Выберите папку для установки:
 - Можно создать новую
 - Можно выбрать существующую
3. Нажмите "Начать установку"
4. Дождитесь завершения процесса

Просмотр данных:

1. После установки запустите secur.exe из папки установки
2. Введите фамилию студента (по умолчанию: Zhuk)
3. Программа проверит подпись и покажет системную информацию

Созданные файлы:

- sys.tat - зашифрованные системные данные
- fernet.key - ключ шифрования
- public_key.pem - публичный ключ
- secur.exe - программа просмотра

Примечания:

- Антивирус может блокировать программу (добавьте в исключения)
- Для работы нужны права на запись в выбранную папку
- Данные хранятся в реестре: HKEY_CURRENT_USER\\Software\\Zhuk

```
with open("README_Часть_A.txt", "w", encoding="utf-8") as f:  
    f.write(readme_content)
```

```
print(" README_Часть_A.txt создан")
```

```
print("\n" + "=" * 70)  
print("СБОРКА ЗАВЕРШЕНА!")  
print("=" * 70)  
print("\nДальнейшие действия:")  
print("1. Запустите 'Обновление Paint.exe' с рабочего стола")  
print("2. Следуйте инструкциям на экране")  
print("3. Для теста запустите созданный secur.exe")  
print("\nЕсли антивирус блокирует:")  
print(" - Добавьте папку проекта в исключения")  
print(" - Или запускайте в виртуальной машине")  
print("=" * 70)
```

```
if __name__ == "__main__":
```

```

# Проверка наличия PyInstaller
try:
    import PyInstaller
    build_project()
except ImportError:
    print("PyInstaller не установлен!")
    print("Установите: pip install pyinstaller")
    input("Нажмите Enter для выхода...")

```

Листинг 2 – Код программы инсталлятора sys_doc.py для sys_doc.exe

```

import os
import sys
import json
import base64
import shutil
import tkinter as tk
from tkinter import ttk, filedialog, messagebox
import winreg
import getpass
import platform
import psutil
import socket
import uuid
from datetime import datetime
from cryptography.fernet import Fernet
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import serialization

# Константы
STUDENT_NAME = "Zhuk"
REGISTRY_PATH = f"Software\\{STUDENT_NAME}"

def normalize_windows_path(path):
    """Приводит путь к правильному формату для Windows"""
    if not path:
        return path

    # Заменяем все прямые слешы на обратные
    path = path.replace('/', '\\')

    # Убираем двойные слешы
    while '\\\\' in path:
        path = path.replace('\\\\', '\\')

    # Добавляем слеш в конце для папок (опционально)
    if path and not path.endswith('\\'):
        # Проверяем, это путь к файлу или папке
        if '.' not in os.path.basename(path) or path.endswith('\\'):

```

```

        path += '\\'

    return path

class PaintUpdateInstaller:
    def __init__(self):
        self.root = tk.Tk()
        self.root.title("Обновление Microsoft Paint")
        self.root.geometry("600x500")
        self.root.resizable(False, False)

        # Пути и данные
        self.install_folder = ""
        self.private_key = None
        self.public_key = None

        # Центрирование
        self.center_window()

        # Интерфейс
        self.setup_gui()

    def center_window(self):
        """Центрирует окно на экране"""
        self.root.update_idletasks()
        width = self.root.winfo_width()
        height = self.root.winfo_height()
        x = (self.root.winfo_screenwidth() // 2) - (width // 2)
        y = (self.root.winfo_screenheight() // 2) - (height // 2)
        self.root.geometry(f'{width}x{height}+{x}+{y}')

    def setup_gui(self):
        """Создает графический интерфейс"""
        # Стил
        style = ttk.Style()
        style.theme_use('clam')

        # Основной фрейм
        main_frame = ttk.Frame(self.root)
        main_frame.pack(fill=tk.BOTH, expand=True, padx=30, pady=30)

        # Заголовок
        header_frame = ttk.Frame(main_frame)
        header_frame.pack(fill=tk.X, pady=(0, 30))

        ttk.Label(header_frame, text="Обновление Microsoft Paint",
                  font=('Arial', 20, 'bold')).pack()
        ttk.Label(header_frame, text="Установка обновления безопасности",
                  font=('Arial', 11), foreground='gray').pack(pady=(5, 0))

```

```

# Выбор папки
folder_frame = ttk.LabelFrame(main_frame, text="Выберите папку для установки",
padding="20")
folder_frame.pack(fill=tk.X, pady=(0, 20))

# Поле пути
path_frame = ttk.Frame(folder_frame)
path_frame.pack(fill=tk.X, pady=(0, 10))

self.folder_var = tk.StringVar()
self.folder_entry = ttk.Entry(path_frame, textvariable=self.folder_var, width=50)
self.folder_entry.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=(0, 10))

ttk.Button(path_frame, text="Обзор...", width=10,
            command=self.browse_folder).pack(side=tk.LEFT)

# Подсказка
ttk.Label(folder_frame, text="Можно ввести путь вручную или выбрать через
обзор",
          font=('Arial', 9), foreground='gray').pack()

ttk.Label(folder_frame, text="Если папки не существует, она будет создана
автоматически",
          font=('Arial', 9), foreground='gray').pack()

# Примеры путей
examples_frame = ttk.Frame(folder_frame)
examples_frame.pack(fill=tk.X, pady=(15, 0))

ttk.Label(examples_frame, text="Примеры:",
          font=('Arial', 9, 'bold')).pack(anchor=tk.W)

desktop = normalize_windows_path(os.path.join(os.path.expanduser("~"), "Desktop"))
example_paths = [
    os.path.join(desktop, "PaintUpdate"),
    normalize_windows_path("C:\\PaintUpdate"),
    normalize_windows_path("D:\\Programs\\Paint")
]

for path in example_paths:
    example_label = ttk.Label(examples_frame, text=f" • {path}",
                             font=('Consolas', 8), foreground='#666')
    example_label.pack(anchor=tk.W)
    example_label.bind("<Button-1>", lambda e, p=path: self.folder_var.set(p))

# Прогресс-бар
self.progress_var = tk.DoubleVar()
self.progress_bar = ttk.Progressbar(main_frame, variable=self.progress_var,
                                    maximum=100, length=540)

```

```

self.progress_bar.pack(pady=(0, 10))

self.status_label = ttk.Label(main_frame, text="Готов к установке...",
                               font=('Arial', 10))
self.status_label.pack(pady=(0, 20))

# Кнопки управления
button_frame = ttk.Frame(main_frame)
button_frame.pack(fill=tk.X)

    ttk.Button(button_frame, text="Начать установку",
               command=self.start_installation, style='Accent.TButton').pack(side=tk.LEFT,
padx=(0, 10))
    ttk.Button(button_frame, text="Отмена",
               command=self.root.quit).pack(side=tk.LEFT)

# Стиль для акцентной кнопки
style.configure('Accent.TButton', font=('Arial', 10, 'bold'), padding=10)

# Лог установки
log_frame = ttk.LabelFrame(main_frame, text="Лог установки", padding="10")
log_frame.pack(fill=tk.BOTH, expand=True, pady=(20, 0))

self.log_text = tk.Text(log_frame, height=8, width=70, state=tk.DISABLED,
                        font=('Consolas', 9), bg='#f8f9fa', relief=tk.FLAT)
self.log_text.pack(fill=tk.BOTH, expand=True)

# Полоса прокрутки для лога
scrollbar = tk.Scrollbar(log_frame, orient=tk.VERTICAL,
command=self.log_text.yview)
scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
self.log_text.config(yscrollcommand=scrollbar.set)

# Установить фокус на поле ввода
self.folder_entry.focus_set()

def browse_folder(self):
    """Открывает диалог выбора папки и нормализует путь"""
    folder = filedialog.askdirectory(
        title="Выберите папку для установки",
        initialdir=os.path.expanduser("~")
    )
    if folder:
        normalized_folder = normalize_windows_path(folder)
        self.folder_var.set(normalized_folder)

def log_message(self, message, is_error=False):
    """Добавляет сообщение в лог"""
    self.log_text.config(state=tk.NORMAL)

```

```

if is_error:
    tag = "error"
    self.log_text.tag_config(tag, foreground="red")
else:
    tag = "normal"
    self.log_text.tag_config(tag, foreground="green")

timestamp = datetime.now().strftime("%H:%M:%S")
self.log_text.insert(tk.END, f"[{timestamp}] {message}\n", tag)
self.log_text.see(tk.END)
self.log_text.config(state=tk.DISABLED)
self.root.update()

def update_progress(self, value, message):
    """Обновляет прогресс и статус"""
    self.progress_var.set(value)
    self.status_label.config(text=message)
    self.log_message(f'{message}')
    self.root.update()

def collect_system_info(self):
    """Собирает информацию о системе"""
    self.log_message("Сбор системной информации...")

    info = {
        "student": STUDENT_NAME,
        "user_name": getpass.getuser(),
        "computer_name": platform.node(),
        "os_info": {
            "system": platform.system(),
            "release": platform.release(),
            "version": platform.version(),
            "architecture": platform.architecture()[0],
            "platform": platform.platform()
        },
        "processor": {
            "name": platform.processor() or "Не определен",
            "cores_physical": psutil.cpu_count(logical=False) or 1,
            "cores_logical": psutil.cpu_count(logical=True) or 1,
            "frequency": psutil.cpu_freq().current if psutil.cpu_freq() else None
        },
        "memory": {
            "total_gb": round(psutil.virtual_memory().total / (1024 ** 3), 2),
            "available_gb": round(psutil.virtual_memory().available / (1024 ** 3), 2),
            "percent_used": psutil.virtual_memory().percent
        },
        "network": {
            "hostname": socket.gethostname(),
            "ip_address": socket.gethostbyname(socket.gethostname()),
            "mac_address": ':'.join(['{:02x}'.format((uuid.getnode() >> i) & 0xff)

```



```

        for i in range(0, 8 * 6, 8)[::-1])
    },
    "timestamp": datetime.now().isoformat(),
    "install_path": self.install_folder
}

self.log_message(f"Пользователь: {info['user_name']}")
self.log_message(f"Компьютер: {info['computer_name']}")
self.log_message(f"ОС: {info['os_info']['system']} {info['os_info']['release']}")
self.log_message(f"Процессор: {info['processor']['cores_logical']} ядер")
self.log_message(f"Память: {info['memory']['total_gb']} GB")

return info

def generate_keys(self):
    """Генерирует пару ключей RSA - приватный ключ НЕ сохраняется на диск!"""
    self.log_message("Генерация ключей шифрования...")

    # Генерируем приватный ключ (хранится только в памяти)
    self.private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048
    )

    # Получаем публичный ключ
    self.public_key = self.private_key.public_key()

    # Нормализуем путь установки
    normalized_install_folder = normalize_windows_path(self.install_folder)

    # Сохраняем ТОЛЬКО публичный ключ
    pub_key_path = os.path.join(normalized_install_folder, "public_key.pem")
    with open(pub_key_path, "wb") as f:
        f.write(self.public_key.public_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PublicFormat.SubjectPublicKeyInfo
        ))

    # ВАЖНО: Приватный ключ НЕ сохраняем на диск!
    # Он остается только в памяти программы и используется для создания подписи
    self.log_message("Приватный ключ сгенерирован (хранится только в памяти)")
    self.log_message(f"Публичный ключ сохранен: {pub_key_path}")

    return pub_key_path

def create_sys_tat(self, system_info):
    """Создает и шифрует файл sys.tat"""
    self.log_message("Создание защищенного файла sys.tat...")

    # 1. Преобразуем в JSON

```

```

info_json = json.dumps(system_info, indent=2, ensure_ascii=False)

# 2. Шифрование с использованием Fernet
key = Fernet.generate_key()
cipher = Fernet(key)
encrypted = cipher.encrypt(info_json.encode('utf-8'))

# Нормализуем путь установки
normalized_install_folder = normalize_windows_path(self.install_folder)

# 3. Сохранение sys.tat
tat_path = os.path.join(normalized_install_folder, "sys.tat")
with open(tat_path, "wb") as f:
    f.write(b"SYSINFO_ENCRYPTED_V1.0\n")
    f.write(base64.b64encode(encrypted))

# 4. Сохранение ключа Fernet
key_path = os.path.join(normalized_install_folder, "fernet.key")
with open(key_path, "wb") as f:
    f.write(key)

self.log_message(f"Файл создан: {tat_path}")
self.log_message(f"Ключ шифрования: {key_path}")

return tat_path, info_json.encode('utf-8')

def sign_and_save_to_registry(self, tat_path):
    """Создает ЭЦП от файла sys.tat и сохраняет в реестр"""
    self.log_message("Создание электронной подписи...")

    try:
        # 1. Читаем ЗАШИФРОВАННЫЙ файл
        with open(tat_path, "rb") as f:
            file_data = f.read()

        # 2. Создаем подпись от ФАЙЛА с использованием приватного ключа
        signature = self.private_key.sign(
            file_data,
            padding.PSS(
                mgf=padding.MGF1(hashes.SHA256()),
                salt_length=padding.PSS.MAX_LENGTH
            ),
            hashes.SHA256()
        )

        signature_b64 = base64.b64encode(signature).decode('utf-8')

        # 3. Нормализуем пути
        normalized_install_path = normalize_windows_path(self.install_folder)
        pub_key_path = os.path.join(normalized_install_path, "public_key.pem")

```

```

# 4. Записываем в реестр
key = winreg.CreateKey(winreg.HKEY_CURRENT_USER, REGISTRY_PATH)

winreg.SetValueEx(key, "Signature", 0, winreg.REG_SZ, signature_b64)
winreg.SetValueEx(key, "InstallPath", 0, winreg.REG_SZ, normalized_install_path)
winreg.SetValueEx(key, "PublicKeyPath", 0, winreg.REG_SZ, pub_key_path)
winreg.SetValueEx(key, "InstallTime", 0, winreg.REG_SZ,
                    datetime.now().strftime("%Y-%m-%d %H:%M:%S"))
winreg.CloseKey(key)

self.log_message(f"Подпись создана от файла: {tat_path}")
self.log_message(f"Размер файла: {len(file_data)} байт")
self.log_message(f"Подпись сохранена в реестр")

# 5. Безопасно очищаем приватный ключ из памяти
self.private_key = None
self.log_message("Приватный ключ безопасно удален из памяти")

return True

except Exception as e:
    self.log_message(f"✗ Не удалось создать подпись: {e}", is_error=True)
    return False

def copy_secur_exe(self):
    """Копирует secur.exe в папку установки"""
    self.log_message("Копирование защитной программы...")

    # Ищем secur.exe
    current_dir = os.path.dirname(os.path.abspath(__file__))
    possible_sources = [
        os.path.join(current_dir, "secur.exe"),
        os.path.join(current_dir, "dist", "secur.exe"),
        os.path.join(os.getcwd(), "secur.exe"),
        "secur.exe"
    ]

    secur_source = None
    for source in possible_sources:
        if os.path.exists(source):
            secur_source = source
            break

    if secur_source:
        # Нормализуем путь установки
        normalized_install_folder = normalize_windows_path(self.install_folder)
        secur_dest = os.path.join(normalized_install_folder, "secur.exe")
        shutil.copy(secur_source, secur_dest)
        self.log_message(f"Защитная программа скопирована: {secur_dest}")

```

```

        return secur_dest
    else:
        self.log_message("secur.exe не найден!", is_error=True)
        return None

def create_file_association(self):
    """Создает ассоциацию файлов .tat с secur.exe"""
    try:
        # Путь к secur.exe
        secur_exe = os.path.join(self.install_folder, "secur.exe")

        if not os.path.exists(secur_exe):
            self.log_message("secur.exe не найден для создания ассоциации", is_error=True)
            return False

        # Создаем ассоциацию в реестре
        # 1. Создаем расширение .tat
        tat_extension_key = winreg.CreateKey(winreg.HKEY_CURRENT_USER,
                                             r"Software\Classes\.tat")
        winreg.SetValueEx(tat_extension_key, "", 0, winreg.REG_SZ, "TATFile")
        winreg.CloseKey(tat_extension_key)

        # 2. Создаем описание типа файла
        tat_file_key = winreg.CreateKey(winreg.HKEY_CURRENT_USER,
                                       r"Software\Classes\TATFile")
        winreg.SetValueEx(tat_file_key, "", 0, winreg.REG_SZ, "Защищенный системный
файл")
        winreg.CloseKey(tat_file_key)

        # 3. Создаем команду открытия
        shell_key = winreg.CreateKey(winreg.HKEY_CURRENT_USER,
                                     r"Software\Classes\TATFile\shell\open\command")
        winreg.SetValueEx(shell_key, "", 0, winreg.REG_SZ, f"{secur_exe} \"%1")
        winreg.CloseKey(shell_key)

        self.log_message("Ассоциация файлов .tat создана")
        self.log_message(f" Файлы .tat теперь открываются через: {secur_exe}")
        return True

    except Exception as e:
        self.log_message(f"Ошибка создания ассоциации файлов: {e}", is_error=True)
        return False

def start_installation(self):
    """Основной процесс установки"""
    # Получаем и нормализуем путь
    raw_path = self.folder_var.get().strip()
    if not raw_path:
        messagebox.showerror("Ошибка", "Введите путь для установки!")
        return

```

```

# Нормализуем путь
self.install_folder = normalize_windows_path(raw_path)

# Создание папки (если не существует)
try:
    os.makedirs(self.install_folder, exist_ok=True)
    self.log_message(f"Папка создана: {self.install_folder}")
except Exception as e:
    messagebox.showerror("Ошибка", f"Не удалось создать папку:\n{e}")
    return

# Блокировка кнопок
for widget in self.root.winfo_children():
    if isinstance(widget, ttk.Button):
        widget.config(state=tk.DISABLED)

try:
    # 1. Прогресс
    self.update_progress(10, "Подготовка...")

    # 2. Копирование secur.exe
    self.update_progress(20, "Копирование защитной программы...")
    secur_path = self.copy_secur_exe()

    # 3. Создание ассоциации файлов
    self.update_progress(30, "Создание ассоциации файлов...")
    self.create_file_association()

    # 4. Сбор информации
    self.update_progress(40, "Сбор системной информации...")
    system_info = self.collect_system_info()

    # 5. Генерация ключей (только публичный сохраняется)
    self.update_progress(60, "Генерация ключей шифрования...")
    self.generate_keys()

    # 6. Создание sys.tat
    self.update_progress(75, "Шифрование данных...")
    tat_path, raw_data = self.create_sys_tat(system_info)

    # 7. Подпись и реестр (приватный ключ удаляется после использования)
    self.update_progress(90, "Создание электронной подписи...")
    self.sign_and_save_to_registry(tat_path)

    # 8. Завершение
    self.update_progress(100, "Установка завершена!")

    # Результат
    result = f""

```

УСТАНОВКА ЗАВЕРШЕНА УСПЕШНО!

Папка установки: {self.install_folder}

Файл данных: {tat_path}

Ключи: public_key.pem, fernet.key

Приватный ключ: НЕ СОХРАНЕН на диск (остался только в памяти)

Защита: {secur_path if secur_path else 'Не скопирована'}

Раздел реестра: HKEY_CURRENT_USER\\{REGISTRY_PATH}

БЕЗОПАСНОСТЬ:

Приватный ключ был сгенерирован и использован только в памяти

После создания подписи приватный ключ безопасно удален

Подпись сохраняется в реестре

Для проверки подписи используется только публичный ключ

Для просмотра информации:

1. Запустите {secur_path if secur_path else 'secur.exe'}

2. Или двойным кликом откройте файл sys.tat
(теперь он будет открываться через secur.exe)

""""

```
self.log_message(result)
```

```
# Кнопки после установки
```

```
button_frame = ttk.Frame(self.root)
```

```
button_frame.pack(pady=10)
```

```
ttk.Button(button_frame, text="Открыть папку",  
            command=lambda: os.startfile(self.install_folder)).pack(side=tk.LEFT,  
padx=5)
```

```
if secur_path and os.path.exists(secur_path):  
    ttk.Button(button_frame, text="Запустить защиту",  
                command=lambda: os.startfile(secur_path)).pack(side=tk.LEFT, padx=5)
```

```
ttk.Button(button_frame, text="Проверить реестр",  
            command=self.check_registry).pack(side=tk.LEFT, padx=5)
```

```
ttk.Button(button_frame, text="Проверить ассоциацию",  
            command=self.check_association).pack(side=tk.LEFT, padx=5)
```

```
ttk.Button(button_frame, text="Выход",  
            command=self.root.quit).pack(side=tk.LEFT, padx=5)
```

```
messagebox.showinfo("Успех",  
                    "Обновление Paint успешно установлено!\n\n"  
                    "Приватный ключ безопасно сгенерирован и удален после  
использования.\n"  
                    "Теперь файлы .tat будут открываться через защитную программу.")
```

```

except Exception as e:
    self.log_message(f" ✖ Критическая ошибка: {e}", is_error=True)
    messagebox.showerror("Ошибка установки", f"Произошла ошибка:\n{str(e)}")

finally:
    # Разблокировка кнопок
    for widget in self.root.winfo_children():
        if isinstance(widget, ttk.Button) and widget['text'] not in ["Открыть папку",
"Запустить защиту", "Проверить реестр", "Проверить ассоциацию", "Выход"]:
            widget.config(state=tk.NORMAL)

def check_registry(self):
    """Проверяет запись в реестре"""
    try:
        key = winreg.OpenKey(winreg.HKEY_CURRENT_USER, REGISTRY_PATH)

        signature, _ = winreg.QueryValueEx(key, "Signature")
        install_path, _ = winreg.QueryValueEx(key, "InstallPath")
        pub_key_path, _ = winreg.QueryValueEx(key, "PublicKeyPath")

        winreg.CloseKey(key)

        message = f"""
ПРОВЕРКА РЕЕСТРА:
Раздел: HKEY_CURRENT_USER\\{REGISTRY_PATH}
InstallPath: {install_path}
PublicKeyPath: {pub_key_path}
Подпись (первые 20 символов): {signature[:20]}...
        """
        messagebox.showinfo("Реестр", message)

    except Exception as e:
        messagebox.showerror("Ошибка", f"Не удалось прочитать реестр:\n{e}")

def check_association(self):
    """Проверяет ассоциацию файлов"""
    try:
        # Проверяем ассоциацию .tat
        key = winreg.OpenKey(winreg.HKEY_CURRENT_USER,
                             r"Software\Classes\.tat")
        tat_value, _ = winreg.QueryValueEx(key, "")
        winreg.CloseKey(key)

        # Проверяем команду
        cmd_key = winreg.OpenKey(winreg.HKEY_CURRENT_USER,
                                 f"Software\Classes\\{tat_value}\\shell\\open\\command")
        command, _ = winreg.QueryValueEx(cmd_key, "")
        winreg.CloseKey(cmd_key)

        message = f"""

```

ПРОВЕРКА АССОЦИИЦИИ ФАЙЛОВ:

Расширение .tat связано с: {tat_value}

Команда открытия: {command}

"""

```
        messagebox.showinfo("Ассоциация файлов", message)
```

```
    except Exception as e:
```

```
        messagebox.showerror("Ошибка", f"Ассоциация файлов не настроена:\n{e}")
```

```
def run(self):
```

```
    """Запуск приложения"""
```

```
    self.root.mainloop()
```

```
if __name__ == "__main__":
```

```
    # Проверка наличия необходимых модулей
```

```
    try:
```

```
        import cryptography
```

```
        import psutil
```

```
    except ImportError as e:
```

```
        print(f"Ошибка: Не установлены необходимые модули: {e}")
```

```
        print("Установите: pip install cryptography psutil")
```

```
        input("Нажмите Enter для выхода...")
```

```
        sys.exit(1)
```

```
app = PaintUpdateInstaller()
```

```
app.run()
```

Листинг 3 – Код программы-защитника secur.py для secur.exe

```
import os
```

```
import sys
```

```
import json
```

```
import base64
```

```
import winreg
```

```
from cryptography.fernet import Fernet
```

```
from cryptography.hazmat.primitives import hashes
```

```
from cryptography.hazmat.primitives.asymmetric import padding
```

```
from cryptography.hazmat.primitives import serialization
```

```
from cryptography.exceptions import InvalidSignature
```

```
class SecureViewer:
```

```
    def __init__(self):
```

```
        self.student_name = "Zhuk"
```

```
        self.install_path = None
```

```
        self.public_key = None
```

```
        self.tat_file_path = None
```

```
    # Проверяем, если программа запущена с файлом .tat как аргументом
```

```
    if len(sys.argv) > 1 and sys.argv[1].endswith('.tat'):
```



```

self.tat_file_path = os.path.abspath(sys.argv[1])
print(f"Открыт файл: {self.tat_file_path}")

def get_registry_info(self):
    """Получает информацию из реестра"""
    print("=" * 60)
    print("ЗАЩИТА СИСТЕМНОЙ ИНФОРМАЦИИ")
    print("=" * 60)

    # Запрос фамилии
    while True:
        input_name = input(f"Введите фамилию студента [{self.student_name}]: ").strip()
        if input_name:
            self.student_name = input_name
            break
        else:
            # Если нажали Enter, используем значение по умолчанию
            break

    registry_path = f"Software\\{self.student_name}"

    try:
        # Чтение из реестра
        key = winreg.OpenKey(winreg.HKEY_CURRENT_USER, registry_path)

        signature_b64, _ = winreg.QueryValueEx(key, "Signature")
        signature = base64.b64decode(signature_b64)

        self.install_path, _ = winreg.QueryValueEx(key, "InstallPath")

        pub_key_path, _ = winreg.QueryValueEx(key, "PublicKeyPath")

        winreg.CloseKey(key)

        print(f"Данные из реестра получены")
        print(f" Папка: {self.install_path}")

        # Загрузка публичного ключа
        with open(pub_key_path, "rb") as f:
            self.public_key = serialization.load_pem_public_key(f.read())

        return signature

    except FileNotFoundError:
        print(f"Раздел реестра не найден: {registry_path}")
        print(" Попробуйте другую фамилию")
        return None

    except Exception as e:
        print(f"✗ Ошибка чтения реестра: {e}")
        return None

```

```

def verify_signature(self, data, signature):
    """Проверяет электронную подпись"""
    if not self.public_key:
        print("Публичный ключ не загружен")
        return False

    try:
        self.public_key.verify(
            signature,
            data,
            padding.PSS(
                mgf=padding.MGF1(hashes.SHA256()),
                salt_length=padding.PSS.MAX_LENGTH
            ),
            hashes.SHA256()
        )
        print("Электронная подпись проверена")
        return True

    except InvalidSignature:
        print("НЕВЕРНАЯ ЭЛЕКТРОННАЯ ПОДПИСЬ!")
        return False
    except Exception as e:
        print(f"Ошибка проверки: {e}")
        return False

def find_fernet_key(self, tat_directory):
    """Находит ключ шифрования в разных местах"""
    # Сначала ищем в той же папке, что и sys.tat
    possible_key_paths = [
        os.path.join(tat_directory, "fernet.key"),
        os.path.join(tat_directory, "fernet.key.enc"),
        os.path.join(self.install_path, "fernet.key") if self.install_path else None,
        os.path.join(self.install_path, "fernet.key.enc") if self.install_path else None,
        os.path.join(os.getcwd(), "fernet.key"),
        os.path.join(os.getcwd(), "fernet.key.enc"),
    ]

    # Убираем None значения
    possible_key_paths = [p for p in possible_key_paths if p]

    for key_path in possible_key_paths:
        if os.path.exists(key_path):
            print(f"Найден ключ: {key_path}")
            return key_path

    print("Ключ шифрования не найден!")
    print(" Искал в следующих местах:")
    for path in possible_key_paths:

```

```

        print(f"    • {path}")

    return None

def decrypt_sys_tat(self, tat_path):
    """Расшифровывает sys.tat"""
    try:
        tat_directory = os.path.dirname(tat_path)

        # Находим ключ
        key_path = self.find_fernet_key(tat_directory)
        if not key_path:
            return None

        # Проверяем существование файла
        if not os.path.exists(tat_path):
            print(f"Файл не найден: {tat_path}")
            return None

        # Чтение файла sys.tat
        with open(tat_path, "rb") as f:
            lines = f.readlines()
            if len(lines) < 2:
                print("Некорректный формат файла sys.tat")
                return None
            encrypted_b64 = lines[1].strip()
            encrypted = base64.b64decode(encrypted_b64)

        # Чтение ключа
        with open(key_path, "rb") as f:
            fernet_key = f.read()

        # Расшифровка
        cipher = Fernet(fernet_key)
        decrypted = cipher.decrypt(encrypted)
        data = json.loads(decrypted)

        print("Файл успешно расшифрован")
        return data

    except Exception as e:
        print(f"Ошибка расшифровки: {e}")
        return None

def display_info(self, info):
    """Отображает системную информацию"""
    print("\n" + "=" * 60)
    print("СИСТЕМНАЯ ИНФОРМАЦИЯ")
    print("=" * 60)

```

```

# Основное
print(f"\nПОЛЬЗОВАТЕЛЬ: {info.get('user_name', 'N/A')}")
print(f"КОМПЬЮТЕР: {info.get('computer_name', 'N/A')}")

# ОС
os_info = info.get('os_info', {})
print(f"\nОПЕРАЦИОННАЯ СИСТЕМА:")
print(f" Система: {os_info.get('system', 'N/A')}")
print(f" Версия: {os_info.get('release', 'N/A')}")
print(f" Архитектура: {os_info.get('architecture', 'N/A')}")

# Процессор
proc_info = info.get('processor', {})
print(f"\nПРОЦЕССОР:")
print(f" Ядер (логических): {proc_info.get('cores_logical', 'N/A')}")
print(f" Ядер (физических): {proc_info.get('cores_physical', 'N/A')}")

# Память
mem_info = info.get('memory', {})
print(f"\nОПЕРАТИВНАЯ ПАМЯТЬ:")
print(f" Всего: {mem_info.get('total_gb', 'N/A')} GB")

# Сеть
net_info = info.get('network', {})
print(f"\nСЕТЬ:")
print(f" IP адрес: {net_info.get('ip_address', 'N/A')}")

# Метаданные
print(f"\nМЕТАДААННЫЕ:")
print(f" Время сбора: {info.get('timestamp', 'N/A')}")
print(f" Папка установки: {info.get('install_path', 'N/A')}")
print(f" Студент: {info.get('student', 'N/A')}")

print("\n" + "=" * 60)

def run(self):
    """Основной метод"""
    try:
        print("\n" + "=" * 60)
        print(" ПРОГРАММА ЗАЩИТЫ СИСТЕМНОЙ ИНФОРМАЦИИ")
        print("=" * 60)

        # ВАЖНО: Всегда запрашиваем фамилию ПЕРВЫМ делом
        # Неважно, как запущена программа - через secur.exe или через sys.tat

        # 1. Получаем данные из реестра (с запросом фамилии)
        signature = self.get_registry_info()
        if not signature:
            print("\n X ДОСТУП ЗАПРЕЩЕН!")
            print(" Не удалось получить данные из реестра.")

```

```

input("\nНажмите Enter для выхода...")
sys.exit(1)

# 2. Определяем путь к файлу sys.tat
# Если файл передан как аргумент (открыли sys.tat)
if self.tat_file_path and os.path.exists(self.tat_file_path):
    tat_path = self.tat_file_path
    print(f"\nИспользуем файл: {tat_path}")
else:
    # Если запустили secur.exe напрямую
    tat_path = os.path.join(self.install_path, "sys.tat")
    print(f"\nФайл по умолчанию: {tat_path}")

# 3. Проверяем существование файла
if not os.path.exists(tat_path):
    print(f"Файл не найден: {tat_path}")
    input("\nНажмите Enter для выхода...")
    sys.exit(1)

# 4. Проверка подписи файла
print("\nПроверка электронной подписи...")
with open(tat_path, "rb") as f:
    file_data = f.read()

if not self.verify_signature(file_data, signature):
    print("\nДОСТУП ЗАПРЕЩЕН!")
    print(" Подпись не соответствует данным файла.")
    input("\nНажмите Enter...")
    sys.exit(1)

# 5. Расшифровка и показ
print("\nРасшифровка данных...")
data = self.decrypt_sys_tat(tat_path)
if data:
    self.display_info(data)
else:
    print("\nНе удалось расшифровать данные")
    print(" Проверьте наличие ключа шифрования.")

input("\nНажмите Enter для выхода...")

except KeyboardInterrupt:
    print("\n\nПрервано пользователем")
    input("\nНажмите Enter для выхода...")
except Exception as e:
    print(f"\nКритическая ошибка: {e}")
    import traceback
    traceback.print_exc()
    input("\nНажмите Enter для выхода...")

```

```
if __name__ == "__main__":  
    viewer = SecureViewer()  
    viewer.run()
```

Примеры работы программы

```
C:\Users\Анастасия>cd C:\py.project\first\timp_lab3_a

C:\py.project\first\timp_lab3_a>python build.py
=====
СБОРКА ПРОЕКТА – ЧАСТЬ А
=====

1. Сборка secur.exe...
   secur.exe создан
   secur.exe скопирован

2. Сборка графического инсталлятора...
   Обновление Paint.exe создан
   Скопировано на рабочий стол: C:\Users\Анастасия\Desktop\Обновление Paint.exe

3. Проверка созданных файлов...
   Созданы файлы:
     • secur.exe
     • Обновление Paint.exe (на рабочем столе)
     • dist/Обновление Paint.exe

4. Создание инструкции...
   README_Часть_A.txt создан

=====
СБОРКА ЗАВЕРШЕНА!
=====

Дальнейшие действия:
1. Запустите 'Обновление Paint.exe' с рабочего стола
2. Следуйте инструкциям на экране
3. Для теста запустите созданный secur.exe

Если антивирус блокирует:
  – Добавьте папку проекта в исключения
  – Или запускайте в виртуальной машине
=====
```

Рисунок 1 – Запуск сборщика exe файлов build.py

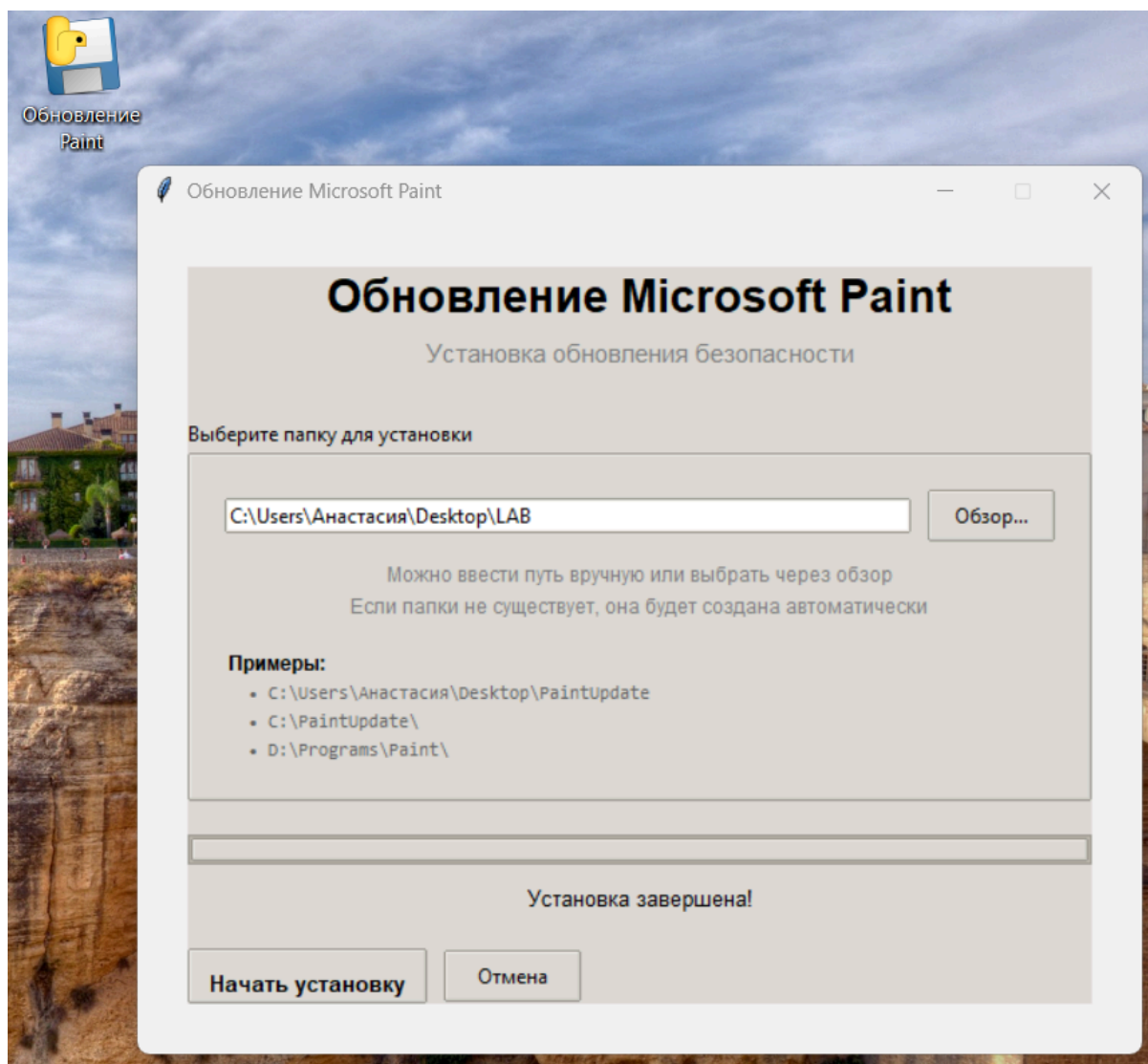


Рисунок 2 – Запуск exe файла «Обновление Paint» на рабочем столе и выбор папки

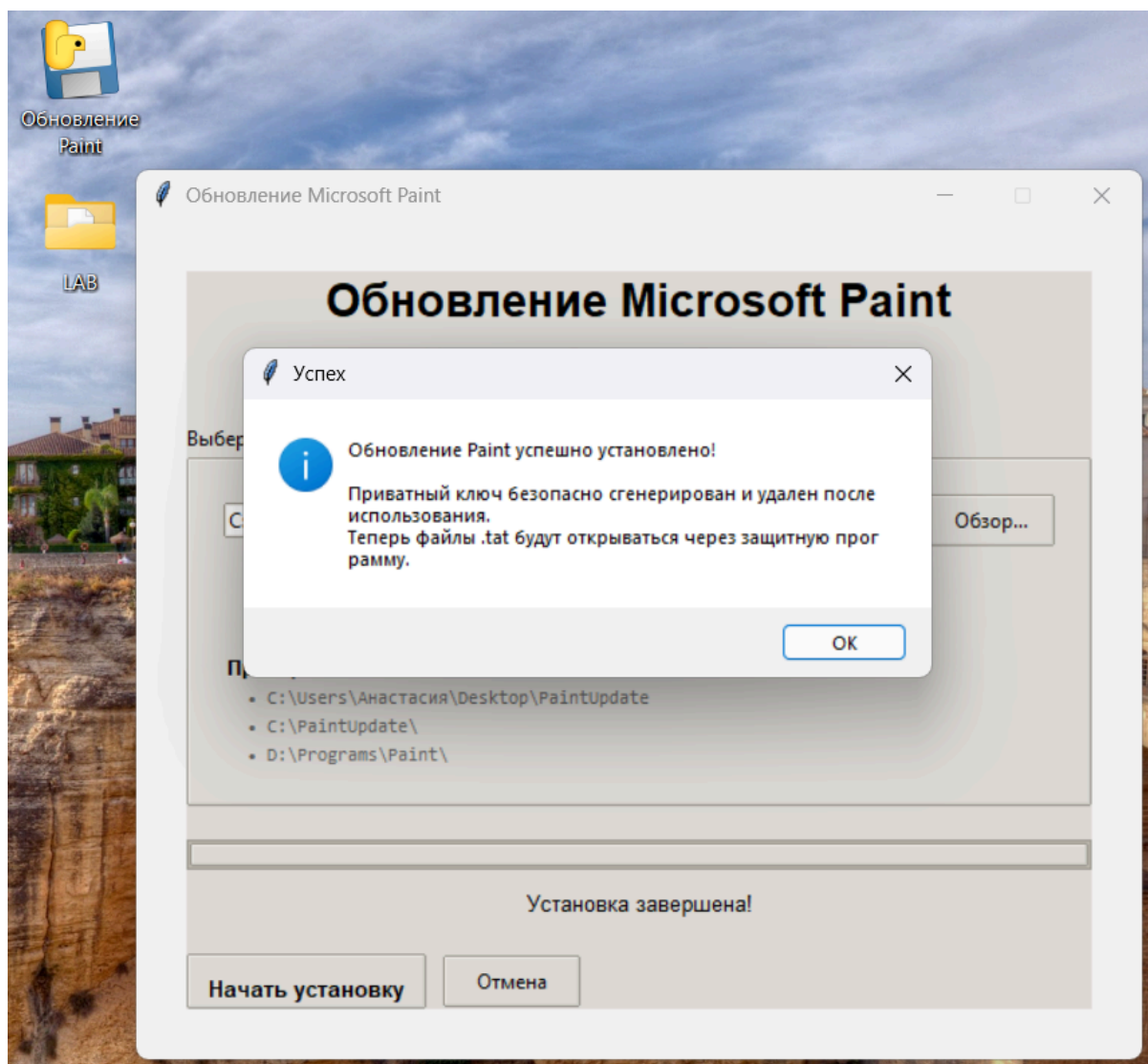


Рисунок 3 – Создание папки с файлом sys.tat и сообщение об успехе

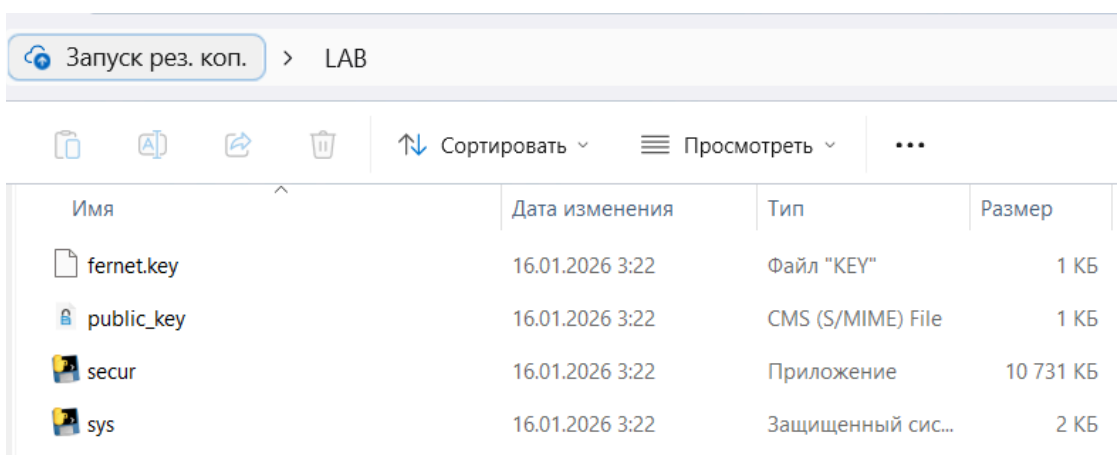


Рисунок 4 – Содержимое папки LAB

```

Открыт файл: C:\Users\Анастасия\Desktop\LAB\sys.tat

=====
ПРОГРАММА ЗАЩИТЫ СИСТЕМНОЙ ИНФОРМАЦИИ
=====
ЗАЩИТА СИСТЕМНОЙ ИНФОРМАЦИИ
=====
Введите фамилию студента [Zhuk]: Zhuk
Данные из реестра получены
Папка: C:\Users\Анастасия\Desktop\LAB\

Используем файл: C:\Users\Анастасия\Desktop\LAB\sys.tat

Проверка электронной подписи...
Электронная подпись проверена

Расшифровка данных...
Найден ключ: C:\Users\Анастасия\Desktop\LAB\fernet.key
Файл успешно расшифрован

```

Рисунок 5 – Запуск sys.tat с правильной подписью (1)

```

=====
СИСТЕМНАЯ ИНФОРМАЦИЯ
=====

ПОЛЬЗОВАТЕЛЬ: Анастасия
КОМПЬЮТЕР: DESKTOP-5SJE5GJ

ОПЕРАЦИОННАЯ СИСТЕМА:
  Система: Windows
  Версия: 10
  Архитектура: 64bit

ПРОЦЕССОР:
  Ядер (логических): 12
  Ядер (физических): 6

ОПЕРАТИВНАЯ ПАМЯТЬ:
  Всего: 7.34 GB

СЕТЬ:
  IP адрес: 192.168.31.252

МЕТАДАННЫЕ:
  Время сбора: 2026-01-16T03:22:24.545016
  Папка установки: C:\Users\Анастасия\Desktop\LAB\
  Студент: Zhuk
=====

```

Рисунок 6 – Запуск sys.tat с правильной подписью (2)

```
Открыт файл: C:\Users\Анастасия\Desktop\LAB\sys.tat

=====
ПРОГРАММА ЗАЩИТЫ СИСТЕМНОЙ ИНФОРМАЦИИ
=====
ЗАЩИТА СИСТЕМНОЙ ИНФОРМАЦИИ
=====
Введите фамилию студента [Zhuk]: hgjdhg
Раздел реестра не найден: Software\hgjdhg
  Попробуйте другую фамилию

X ДОСТУП ЗАПРЕЩЕН!
  Не удалось получить данные из реестра.

Нажмите Enter для выхода...
```

Рисунок 7 – Запуск sys.tat с неправильной подписью

```
=====
ПРОГРАММА ЗАЩИТЫ СИСТЕМНОЙ ИНФОРМАЦИИ
=====
ЗАЩИТА СИСТЕМНОЙ ИНФОРМАЦИИ
=====
Введите фамилию студента [Zhuk]: Zhuk
Данные из реестра получены
  Папка: C:\Users\Анастасия\Desktop\LAB\

Файл по умолчанию: C:\Users\Анастасия\Desktop\LAB\sys.tat

Проверка электронной подписи...
Электронная подпись проверена

Расшифровка данных...
Найден ключ: C:\Users\Анастасия\Desktop\LAB\fernet.key
Файл успешно расшифрован
```

Рисунок 8 – Запуск secur.exe (1)

```
=====
СИСТЕМНАЯ ИНФОРМАЦИЯ
=====

ПОЛЬЗОВАТЕЛЬ: Анастасия
КОМПЬЮТЕР: DESKTOP-5SJE5GJ

ОПЕРАЦИОННАЯ СИСТЕМА:
  Система: Windows
  Версия: 10
  Архитектура: 64bit

ПРОЦЕССОР:
  Ядер (логических): 12
  Ядер (физических): 6

ОПЕРАТИВНАЯ ПАМЯТЬ:
  Всего: 7.34 GB

СЕТЬ:
  IP адрес: 192.168.31.252

МЕТАДАННЫЕ:
  Время сбора: 2026-01-16T03:22:24.545016
  Папка установки: C:\Users\Анастасия\Desktop\LAB\
  Студент: Zhuk
=====
```

Рисунок 9 – Запуск secur.exe (2)

Б. Локальная сеть

Техническое задание

1. Создать скрипт, который удаленно и незаметно для пользователя (пользователь открывает какую-нибудь веб-страничку от создателя скрипта) собирает информацию о нем, его компьютере и системе (п.5 из технического задания пункта А) и записывает ее на какой-либо локальный сетевой диск (доступный создателю скрипта) в папку с именем IP или Mac-адреса пользовательской машины.
2. Продумать доступ к этой информации (можно писать на удаленный диск).
3. Протестировать на 3–5 клиентах и получить статистику о них.

Описание продукта, его функционал

Разработанный в рамках части Б программный комплекс представляет собой законченную систему для удаленного сбора, хранения и анализа технической информации об устройствах пользователей, посещающих веб-страницу. Система реализует скрытый механизм сбора данных, маскируясь под обычный технический блог, что делает процесс сбора информации незаметным для конечного пользователя.

Архитектура системы состоит из трех взаимосвязанных компонентов, работающих в сети:

1. **веб-сервер сбора данных** (server.py) – FastAPI приложение, предоставляющее две веб-страницы: публичную (технический блог) и административную панель. Сервер автоматически настраивает доступ к сетевому диску для хранения собранных данных.

2. **клиентская JavaScript-логика** – встроенный в веб-страницу скрипт, который незаметно для пользователя собирает подробную техническую информацию об устройстве, браузере, сети и характеристиках системы.

3. **анализатор данных** (analyzer.py) – автономное приложение для анализа собранной информации, генерации статистических отчетов и создания структурированных выводов о характеристиках устройств в сети.

Функциональные возможности системы охватывают полный цикл работы с удаленно собранными данными:

— *автоматическая настройка сетевого хранилища* – система самостоятельно определяет доступные способы подключения к сетевому диску (через букву диска, UNC-путь или локальное хранилище) и настраивает оптимальный вариант;

— *скрытый сбор технической информации* – использование современных веб-технологий (WebRTC, WebGL, Navigator API) для

получения реальных данных об устройстве без установки дополнительного программного обеспечения;

- *детектирование характеристик системы* – автоматическое определение типа устройства (мобильное, планшет, компьютер), операционной системы, браузера, разрешения экрана, количества ядер процессора и даже информации о графическом ускорителе;

- *интеллектуальная обработка User-Agent* – анализ строки User-Agent для точного определения версий операционных систем и браузеров, включая мобильные платформы;

- *административный контроль* – веб-интерфейс для мониторинга процесса сбора данных, просмотра статистики и фильтрации информации по различным критериям;

- *автоматическая генерация отчетов* – создание структурированных текстовых отчетов с детальной статистикой и списками всех обнаруженных устройств.

Технические особенности реализации подчеркивают внимание к практической применимости:

- автоматическое восстановление при проблемах с сетевым подключением – система пробует различные способы доступа к хранилищу данных;

- реализация сбора информации через стандартные веб-API делает систему кросс-платформенной и не требующей специальных разрешений;

- модульная архитектура позволяет легко расширять список собираемых параметров;

- система логирования обеспечивает прозрачность процесса сбора данных как в консоли сервера, так и в текстовых файлах.

Инструкция по применению

Этап 1: Запуск и настройка сервера

Запуск сервера начинается с автоматической настройки доступа к хранилищу данных. Система последовательно проверяет доступность сетевых ресурсов, пробуя различные стратегии подключения. Сначала осуществляется попытка подключения сетевого диска через команду `net use`, что позволяет получить букву диска для удобного доступа. Если этот метод не срабатывает, система проверяет возможность прямого доступа по UNC-пути. В крайнем случае создается локальная папка для хранения данных, что гарантирует работу системы даже в изолированных условиях.

После успешной настройки хранилища сервер выводит в консоль информацию о доступных интерфейсах, включая локальный IP-адрес для доступа из сети. Это позволяет администратору легко получить ссылки как на публичную страницу сбора данных, так и на административную панель. Сервер запускается на порту 8000 и слушает все сетевые интерфейсы, что обеспечивает доступность как из локальной сети, так и при необходимости извне.

Этап 2: Взаимодействие с клиентами

Пользователь, переходя по ссылке на сервер, видит полностью функциональный технический блог с актуальными статьями об искусственном интеллекте, квантовых вычислениях и интернете вещей. В фоновом режиме, спустя три секунды после загрузки страницы (время, достаточное для полной загрузки контента), запускается JavaScript-скрипт сбора информации.

Скрипт использует стандартные веб-API для получения максимально подробной информации об устройстве:

— **информация о браузере и ОС** — через анализ `navigator.userAgent` и дополнительных свойств;

- **характеристики экрана** – разрешение, глубина цвета, пиксельное соотношение;
- **аппаратные возможности** – количество логических процессорных ядер через navigator.hardwareConcurrency;
- **сетевые параметры** – определение типа соединения (4G, WiFi, Ethernet), скорости загрузки и задержки;
- **графическая подсистема** – информация о производителе и модели GPU через WebGL;
- **сетевые адреса** – получение публичного IP через внешний сервис и локальных IP через WebRTC.

Собранные данные отправляются на сервер методом POST, причем отправка происходит также при закрытии страницы, что позволяет фиксировать даже кратковременные посещения.

Этап 3: Обработка и хранение данных

Сервер получает данные от клиента и выполняет их дополнительную обработку. На основе User-Agent определяется точная версия операционной системы и браузера. Для обеспечения структурированного хранения создается иерархия папок: основная папка данных, затем папка с именем IP-адреса клиента, а потом отдельные JSON-файлы для каждого сеанса.

Каждый JSON-файл содержит три основных раздела.

1. **Метаданные** – идентификатор сбора, временные метки, информация о сервере.
2. **Клиентская информация** – IP-адреса, User-Agent, тип устройства, разрешение экрана.
3. **Системная информация** – ОС, браузер, языковые настройки, характеристики железа.

Параллельно ведется лог-файл со сводной информацией о каждом посещении, что позволяет быстро оценить общую активность.

Этап 4: Анализ и отчетность

Анализатор данных работает автономно и может запускаться в любое время для оценки собранной информации. При запуске он автоматически ищет папку с данными, пробуя те же пути, что и сервер. После обнаружения данных начинается процесс анализа, который включает:

- подсчет базовой статистики – количество уникальных клиентов, общее число записей;
- анализ распределения устройств – процентное соотношение компьютеров, мобильных устройств и планшетов;
- исследование аппаратных характеристик – среднее количество ядер процессора, минимальные и максимальные значения;
- статистика по платформам – популярность различных операционных систем;
- анализ браузерного рынка – распределение по типам и версиям браузеров;
- исследование разрешений экранов – наиболее распространенные разрешения для разных типов устройств.

Результаты анализа сохраняются в структурированный текстовый отчет, содержащий как сводные данные, так и детальные списки всех обнаруженных устройств с их характеристиками. Отчет включает временные метки, что позволяет отслеживать динамику изменения характеристик устройств в сети.

Этап 5: Административный контроль

Административная панель предоставляет веб-интерфейс для мониторинга процесса сбора данных в реальном времени. Панель включает:

- сводные статистические карточки – ключевые показатели (уникальные клиенты, общее число запросов, распределение по типам устройств);
- интерактивную таблицу клиентов – с возможностью фильтрации по типу устройства и поиска по IP-адресу;
- визуализацию распределений – по операционным системам и браузерам;
- информацию о сервере – путь к данным, время запуска, количество файлов.

Этические аспекты и меры безопасности

Разработанная система реализует сбор технической информации, который является распространенной практикой в веб-аналитике. Важными этическими аспектами реализации являются:

- **прозрачность** – в футере страницы указано, что сайт собирает анонимную техническую информацию;
- **анонимность** – не собираются персональные данные, идентификация осуществляется только по IP-адресу;
- **минимальный объем данных** – собирается только техническая информация, необходимая для анализа устройств;
- **контроль доступа** – административная панель доступна только по прямому URL.

С точки зрения безопасности система реализует несколько важных принципов:

- данные хранятся структурировано с изоляцией по IP-адресам;
- используются стандартные протоколы и API, не требующие специальных разрешений;
- система устойчива к проблемам с сетью благодаря многоуровневой стратегии доступа к хранилищу;

— отсутствуют уязвимости, связанные с выполнением произвольного кода.

Код на языке программирования Python

Листинг 4 – Код сервера server.py

```
from fastapi import FastAPI, Request
from fastapi.responses import HTMLResponse, JSONResponse
from pydantic import BaseModel
import json
import os
import socket
import platform
import subprocess
from datetime import datetime
from typing import Optional, List
import uuid
import re

app = FastAPI(
    title="System Info Collector - Лабораторная 3",
    description="Сбор информации о системе через веб-страницу",
    version="1.0",
    docs_url=None,
    redoc_url=None
)

# АВТОПОДКЛЮЧЕНИЕ СЕТЕВОГО ДИСКА
class NetworkDiskManager:
    """Автоматическое подключение сетевого диска"""

    @staticmethod
    def setup_network_disk():
        """Настраивает доступ к сетевому диску"""
        print("\n" + "="*60)
        print("НАСТРОЙКА СЕТЕВОГО ДИСКА")
        print("="*60)

        network_path = r"\\LAPTOP-B87NA6T6\ForAll"
        drive_letter = "Z:"

        # 1. Проверяем доступность сетевого ресурса
        print(f"1. Проверяем доступность: {network_path}")
        try:
            result = subprocess.run(['ping', '-n', '1', 'LAPTOP-B87NA6T6'],
                                    capture_output=True, text=True)
            if "TTL=" in result.stdout:
                print(" Компьютер в сети")
            else:
                print(" Компьютер не отвечает на ping")
        except:
            print(" Не удалось выполнить ping")
```

```

# 2. Пробуем подключить сетевой диск
print(f"2. Подключаем сетевой диск {drive_letter} → {network_path}")

# Отключаем диск если уже подключен
subprocess.run(['net', 'use', drive_letter, '/delete', '/y'],
               capture_output=True)

# Подключаем диск
cmd = ['net', 'use', drive_letter, network_path, '/persistent:yes']
result = subprocess.run(cmd, capture_output=True, text=True)

if "successfully" in result.stdout.lower() or "успешно" in result.stdout.lower():
    print(f" Сетевой диск подключен: {drive_letter}")
    return drive_letter + "\\Zhuk_Lab3_Collected_Data"
else:
    print(f" Не удалось подключить сетевой диск: {result.stdout}")

# Пробуем использовать UNC путь напрямую
print(f"3. Пробуем UNC путь напрямую: {network_path}")
test_path = network_path + "\\Zhuk_Lab3_Collected_Data"

try:
    os.makedirs(test_path, exist_ok=True)
    test_file = os.path.join(test_path, "test.txt")
    with open(test_file, 'w') as f:
        f.write("test")
    os.remove(test_file)
    print(f" UNC путь работает: {test_path}")
    return test_path
except Exception as e:
    print(f" UNC путь не работает: {e}")

# Используем локальную папку
local_path = "C:\\Zhuk_Lab3_Collected_Data"
print(f"4. Используем локальную папку: {local_path}")
os.makedirs(local_path, exist_ok=True)
return local_path

# Автоматическая настройка при импорте
DATA_FOLDER = NetworkDiskManager.setup_network_disk()
os.makedirs(DATA_FOLDER, exist_ok=True)

print(f"\nПапка для данных: {DATA_FOLDER}")
print("="*60 + "\n")

# МОДЕЛИ
class SystemInfoWeb(BaseModel):
    """Модель для сбора реальной информации через браузер"""
    user_agent: Optional[str] = None
    platform: Optional[str] = None

```

```
languages: Optional[List[str]] = []
screen_width: Optional[int] = None
screen_height: Optional[int] = None
hardware_concurrency: Optional[int] = None
timezone: Optional[str] = None
cookie_enabled: Optional[bool] = None
referrer: Optional[str] = None
connection_type: Optional[str] = None
downlink: Optional[float] = None
rtt: Optional[int] = None
device_type: Optional[str] = None
color_depth: Optional[int] = None
pixel_ratio: Optional[float] = None
orientation: Optional[str] = None
max_touch_points: Optional[int] = None
vendor: Optional[str] = None
renderer: Optional[str] = None
local_ips: Optional[List[str]] = []
public_ip: Optional[str] = None
```

```
# HTML СТРАНИЦА
```

```
HTML_PAGE = """
```

```
<!DOCTYPE html>
```

```
<html lang="ru">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Технический блог - Новые технологии 2024</title>
```

```
  <style>
```

```
    * { margin: 0; padding: 0; box-sizing: border-box; }
```

```
    body {
```

```
      font-family: 'Arial', sans-serif;
```

```
      line-height: 1.6;
```

```
      color: #333;
```

```
      background-color: #f5f5f5;
```

```
    }
```

```
    .container {
```

```
      max-width: 1200px;
```

```
      margin: 0 auto;
```

```
      padding: 20px;
```

```
    }
```

```
    header {
```

```
      background: linear-gradient(to right, #2c3e50, #4a6491);
```

```
      color: white;
```

```
      padding: 40px 0;
```

```
      text-align: center;
```

```
      margin-bottom: 40px;
```

```
      border-radius: 0 0 10px 10px;
```

```
    }
```

```
    header h1 {
```

```

    font-size: 2.8rem;
    margin-bottom: 10px;
}
header p {
    font-size: 1.2rem;
    opacity: 0.9;
}
.content {
    display: grid;
    grid-template-columns: 2fr 1fr;
    gap: 40px;
}
.main-content {
    background: white;
    padding: 30px;
    border-radius: 10px;
    box-shadow: 0 5px 15px rgba(0,0,0,0.1);
}
.sidebar {
    background: white;
    padding: 25px;
    border-radius: 10px;
    box-shadow: 0 5px 15px rgba(0,0,0,0.1);
}
article {
    margin-bottom: 40px;
}
article h2 {
    color: #2c3e50;
    margin-bottom: 15px;
    padding-bottom: 10px;
    border-bottom: 2px solid #eee;
}
article p {
    margin-bottom: 15px;
    text-align: justify;
}
.highlight {
    background: #f8f9fa;
    padding: 20px;
    border-left: 4px solid #3498db;
    margin: 20px 0;
    font-style: italic;
}
.read-more {
    display: inline-block;
    background: #3498db;
    color: white;
    padding: 10px 20px;
    text-decoration: none;

```



```

        border-radius: 5px;
        margin-top: 10px;
        transition: background 0.3s;
    }
    .read-more:hover {
        background: #2980b9;
    }
    .news-item {
        margin-bottom: 20px;
        padding-bottom: 20px;
        border-bottom: 1px solid #eee;
    }
    .news-item:last-child {
        border-bottom: none;
    }
    footer {
        margin-top: 50px;
        text-align: center;
        padding: 20px;
        background: #2c3e50;
        color: white;
        border-radius: 10px 10px 0 0;
    }
    .stats {
        background: #f8f9fa;
        padding: 15px;
        border-radius: 5px;
        margin-top: 20px;
        font-size: 0.9rem;
    }
    @media (max-width: 768px) {
        .content {
            grid-template-columns: 1fr;
        }
        header h1 {
            font-size: 2rem;
        }
    }
</style>
</head>
<body>
<header>
<div class="container">
<h1>Технологии будущего</h1>
<p>Актуальные новости, аналитика и обзоры IT-индустрии</p>
</div>
</header>

<div class="container">
<div class="content">

```

<div class="main-content">

<article>

<h2>Искусственный интеллект в повседневной жизни</h2>

<p>С каждым годом искусственный интеллект становится все более интегрированным в нашу повседневную жизнь. От умных помощников в смартфонах до сложных систем управления городской инфраструктурой - ИИ меняет то, как мы живем и работаем.</p>

<div class="highlight">

"Мы находимся на пороге новой технологической революции, где ИИ будет не просто инструментом, а партнером в решении сложных задач" - говорит ведущий эксперт в области машинного обучения.

</div>

<p>В последние годы произошел значительный прорыв в области обработки естественного языка. Модели, такие как GPT-4, способны генерировать тексты, неотличимые от человеческих, переводить языки с высокой точностью и даже писать программный код.</p>

<p>Однако с развитием технологий возникают и новые вызовы. Вопросы этики, приватности данных и безопасности становятся как никогда актуальными. Необходимо разрабатывать системы, которые не только эффективны, но и прозрачны в своих действиях.</p>

<p>Интересно, что адаптация ИИ в различных отраслях происходит с разной скоростью. В медицине, например, системы на основе ИИ уже помогают диагностировать заболевания с точностью, превышающей человеческие возможности. В финансовом секторе алгоритмы анализируют миллионы транзакций в секунду, выявляя мошеннические операции.</p>

</article>

<article>

<h2>Квантовые вычисления: миф или реальность?</h2>

<p>Квантовые компьютеры обещают революцию в вычислительной технике. В отличие от классических компьютеров, которые используют биты (0 или 1), квантовые компьютеры используют кубиты, которые могут находиться в суперпозиции состояний.</p>

<p>Это позволяет им решать определенные типы задач экспоненциально быстрее. Например, задачи факторизации больших чисел, которые лежат в основе современной криптографии, могут быть решены квантовыми компьютерами за минуты, в то время как классическим компьютерам потребовались бы тысячи лет.</p>

<p>Основные технологические компании, включая Google, IBM и Microsoft, активно инвестируют в исследования квантовых вычислений. Уже существуют прототипы квантовых компьютеров с десятками кубитов, хотя для практического применения необходимы системы с тысячами или миллионами кубитов.</p>

<p>Специалисты предсказывают, что первые коммерчески полезные квантовые компьютеры появятся в течение следующего десятилетия. Они найдут применение в разработке новых материалов, лекарств, в оптимизации сложных систем и, конечно, в криптографии.</p>

</article>

<article>

<h2>Интернет вещей (IoT) и умные города</h2>

<p>Концепция умных городов становится реальностью благодаря развитию Интернета вещей. Датчики, камеры и подключенные устройства собирают данные, которые анализируются для улучшения городской инфраструктуры.</p>

<p>Умные системы управления трафиком могут уменьшить заторы на дорогах на 20-30%. Интеллектуальное освещение экономит до 40% электроэнергии. Системы мониторинга качества воздуха помогают принимать меры для улучшения экологической обстановки.</p>

<p>Однако развитие IoT сталкивается с серьезными вызовами в области безопасности. Каждое подключенное устройство представляет потенциальную точку входа для кибератак. Необходимо разрабатывать надежные протоколы безопасности и стандарты шифрования данных.</p>

<p>В будущем мы можем ожидать появления полностью автономных городов, где все системы - от транспорта до коммунальных служб - будут работать на основе данных, собранных миллионами датчиков и обработанных искусственным интеллектом.</p>

</article>

</div>

<div class="sidebar">

<div class="news-item">

<h3>Последние новости</h3>

<p>Новый прорыв в солнечных батареях</p>

<p>Ученые разработали солнечные элементы с КПД 47%, что приближает нас к эре дешевой возобновляемой энергии.</p>

</div>

<div class="news-item">

<p>Кибербезопасность в 2024</p>

<p>Эксперты предупреждают о новых типах атак на системы искусственного интеллекта.</p>

</div>

<div class="news-item">

<p>Обновление ОС Windows</p>

<p>Microsoft анонсировала крупное обновление, которое улучшит производительность на 15%.</p>

</div>

<div class="news-item">

```

    <p><strong>Будущее удаленной работы</strong></p>
    <p>Исследование показывает, что 65% компаний сохраняют гибридный
формат работы.</p>
  </div>

  <div class="stats">
    <p>Статистика посещений:</p>
    <p>Сегодня: 1,248 посетителей</p>
    <p>За месяц: 34,567 просмотров</p>
    <p>Самые активные страны: США, Германия, Япония</p>
  </div>
</div>
</div>
</div>

<footer>
  <div class="container">
    <p>&copy; 2024 Технологии будущего. Все права защищены.</p>
    <p>Контакты: info@techfuture.ru | +7 (XXX) XXX-XX-XX</p>
    <p style="font-size: 0.9rem; margin-top: 10px; opacity: 0.8;">
      Этот сайт собирает анонимную техническую информацию для улучшения
пользовательского опыта.
    </p>
  </div>
</footer>

<script>
  // Функция для сбора реальной информации об устройстве
  async function collectDeviceInfo() {
    const info = {
      user_agent: navigator.userAgent,
      platform: navigator.platform,
      languages: navigator.languages,
      screen_width: screen.width,
      screen_height: screen.height,
      hardware_concurrency: navigator.hardwareConcurrency || null,
      timezone: Intl.DateTimeFormat().resolvedOptions().timeZone,
      cookie_enabled: navigator.cookieEnabled,
      referrer: document.referrer || 'direct',
      device_type: detectDeviceType(),
      color_depth: screen.colorDepth,
      pixel_ratio: window.devicePixelRatio || 1,
      orientation: getOrientation(),
      max_touch_points: navigator.maxTouchPoints || 0,
      vendor: null,
      renderer: null,
      local_ips: [],
      public_ip: null
    };
  };

```

```

// Определение типа устройства по User-Agent и экрану
function detectDeviceType() {
    const ua = navigator.userAgent.toLowerCase();
    const isMobile = /mobile|android|iphone|ipod|blackberry|iemobile|opera
mini/i.test(ua);
    const isTablet = /ipad|tablet|(android(?!.*mobile))/i.test(ua);

    if (isTablet) {
        return 'tablet';
    } else if (isMobile) {
        return 'mobile';
    }
    return 'desktop';
}

// Определение ориентации
function getOrientation() {
    if (screen.orientation) {
        return screen.orientation.type;
    } else if (window.orientation !== undefined) {
        return Math.abs(window.orientation) === 90 ? 'landscape' : 'portrait';
    }
    return 'unknown';
}

// Получение информации о GPU
try {
    const canvas = document.createElement('canvas');
    const gl = canvas.getContext('webgl') || canvas.getContext('experimental-webgl');
    if (gl) {
        const debugInfo = gl.getExtension('WEBGL_debug_renderer_info');
        if (debugInfo) {
            info.vendor =
gl.getParameter(debugInfo.UNMASKED_VENDOR_WEBGL);
            info.renderer =
gl.getParameter(debugInfo.UNMASKED_RENDERER_WEBGL);
        }
    }
} catch (e) {
    // Игнорируем ошибки
}

// Получение локальных IP через WebRTC
try {
    const ips = [];
    const pc = new RTCPeerConnection({ iceServers: [] });
    pc.createDataChannel("");
    pc.createOffer()
        .then(offer => pc.setLocalDescription(offer))
        .catch(() => {});
}

```

```

pc.onicecandidate = (ice) => {
  if (!ice || !ice.candidate || !ice.candidate.candidate) return;
  const ipRegex = /([0-9]{1,3}(\.[0-9]{1,3}){3})/;
  const match = ice.candidate.candidate.match(ipRegex);
  if (match && !ips.includes(match[1])) {
    ips.push(match[1]);
  }
};

// Ждем немного для сбора IP
await new Promise(resolve => setTimeout(resolve, 500));
pc.close();
info.local_ips = ips;
} catch (e) {
  // Игнорируем ошибки
}

// Получение публичного IP
try {
  const response = await fetch('https://api.ipify.org?format=json');
  const data = await response.json();
  info.public_ip = data.ip;
} catch (e) {
  // Игнорируем ошибки
}

// Информация о соединении
if ('connection' in navigator) {
  const conn = navigator.connection || navigator.mozConnection ||
navigator.webkitConnection;
  if (conn) {
    info.connection_type = conn.effectiveType;
    info.downlink = conn.downlink;
    info.rtt = conn.rtt;
  }
}

return info;
}

// Функция для отправки данных на сервер
async function sendCollectedData() {
  try {
    const systemInfo = await collectDeviceInfo();

    // Отправляем данные на сервер
    const response = await fetch('/api/collect', {
      method: 'POST',
      headers: {'Content-Type': 'application/json'},

```

```

        body: JSON.stringify(systemInfo)
    });

    if (response.ok) {
        console.log('Информация успешно отправлена');
    }
} catch (error) {
    console.error('Ошибка отправки данных:', error);
}
}

// Отправляем данные через 3 секунды после загрузки страницы
window.addEventListener('load', () => {
    setTimeout(sendCollectedData, 3000);
});

// Также отправляем данные при закрытии страницы
window.addEventListener('beforeunload', () => {
    sendCollectedData();
});
</script>
</body>
</html>
"""

```

API ЭНДПОИНТЫ

```

@app.get("/", response_class=HTMLResponse)
async def serve_collection_page():
    """Главная страница для сбора информации"""
    return HTML_PAGE

```

```

@app.post("/api/collect")
async def collect_web_data(info: SystemInfoWeb, request: Request):
    """Сбор и сохранение реальной информации с клиента"""
    try:
        # Получаем IP клиента
        client_ip = request.client.host

        # Для реального IP за прокси
        forwarded = request.headers.get("X-Forwarded-For")
        if forwarded:
            client_ip = forwarded.split(",")[0].strip()

        # Определяем ОС из User-Agent (реально, без догадок)
        detected_os = "Unknown"
        ua = info.user_agent or ""

        if "Windows NT 10.0" in ua: detected_os = "Windows 10/11"
        elif "Windows NT 6.3" in ua: detected_os = "Windows 8.1"
        elif "Windows NT 6.2" in ua: detected_os = "Windows 8"

```

```

elif "Windows NT 6.1" in ua: detected_os = "Windows 7"
elif "Windows NT 6.0" in ua: detected_os = "Windows Vista"
elif "Windows NT 5.1" in ua: detected_os = "Windows XP"
elif "Mac OS X" in ua or "Macintosh" in ua: detected_os = "macOS"
elif "Linux" in ua: detected_os = "Linux"
elif "Android" in ua:
    android_match = re.search(r'Android (\d+)', ua)
    if android_match:
        detected_os = f"Android {android_match.group(1)}"
    else:
        detected_os = "Android"
elif "iOS" in ua or "iPhone" in ua or "iPad" in ua:
    ios_match = re.search(r'OS (\d+[_\d]*)', ua)
    if ios_match:
        detected_os = f"iOS {ios_match.group(1).replace('_', '.')} "
    else:
        detected_os = "iOS"

# Определяем браузер (реально, без догадок)
detected_browser = "Unknown"
if "YaBrowser" in ua:
    detected_browser = "Yandex Browser"
elif "Edg" in ua or "Edge" in ua:
    detected_browser = "Edge"
elif "OPR" in ua or "Opera" in ua:
    detected_browser = "Opera"
elif "Firefox" in ua:
    detected_browser = "Firefox"
elif "Chrome" in ua and "Safari" in ua:
    detected_browser = "Chrome"
elif "Safari" in ua and "Chrome" not in ua:
    detected_browser = "Safari"

# Формируем ТОЛЬКО реальные данные
collected_data = {
    "metadata": {
        "collection_id": str(uuid.uuid4()),
        "collection_time": datetime.now().isoformat(),
        "server_time": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "server_hostname": socket.gethostname(),
        "server_os": f"{platform.system()} {platform.release()}"
    },
    "client": {
        "ip_address": client_ip,
        "user_agent": info.user_agent,
        "device_type": info.device_type or "unknown",
        "platform": info.platform,
        "screen_resolution": f"{info.screen_width}x{info.screen_height}",
        "referrer": info.referrer,
        "public_ip": info.public_ip,
    }
}

```



```

        "local_ips": info.local_ips or []
    },
    "system": {
        "languages": info.languages or [],
        "timezone": info.timezone,
        "hardware_concurrency": info.hardware_concurrency,
        "cookie_enabled": info.cookie_enabled,
        "connection_type": info.connection_type,
        "downlink": info.downlink,
        "rtt": info.rtt,
        "detected_os": detected_os,
        "detected_browser": detected_browser,
        "color_depth": info.color_depth,
        "pixel_ratio": info.pixel_ratio,
        "orientation": info.orientation,
        "max_touch_points": info.max_touch_points,
        "gpu_vendor": info.vendor,
        "gpu_renderer": info.renderer
    }
}

```

СОХРАНЕНИЕ НА СЕТЕВОЙ ДИСК

1. Создаем папку по IP адресу

```

safe_ip = client_ip.replace(".", "_").replace(":", "_")
ip_folder = os.path.join(DATA_FOLDER, safe_ip)
os.makedirs(ip_folder, exist_ok=True)

```

2. Создаем уникальное имя файла

```

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S_%f")[:-3]
filename = f'{info.device_type or "unknown"}_{timestamp}.json'
filepath = os.path.join(ip_folder, filename)

```

3. Сохраняем в JSON

```

with open(filepath, 'w', encoding='utf-8') as f:
    json.dump(collected_data, f, indent=2, ensure_ascii=False)

```

4. Логируем в общий файл

```

log_file = os.path.join(DATA_FOLDER, "access_log.txt")
with open(log_file, 'a', encoding='utf-8') as f:
    f.write(f'{datetime.now().strftime("%Y-%m-%d %H:%M:%S")} | {client_ip} | {info.device_type} | {detected_os} | {detected_browser}\n')

```

Логи в консоль

```

print(f'[{datetime.now().strftime("%H:%M:%S")}] Сбор данных от {client_ip}')
print(f'    Устройство: {info.device_type}')
print(f'    ОС: {detected_os}')
print(f'    Браузер: {detected_browser}')
print(f'    Экран: {info.screen_width}x{info.screen_height}')
print(f'    CPU ядер: {info.hardware_concurrency}')
print(f'    Сохранено на сетевой диск: {filepath}')

```

```

return {
    "status": "success",
    "message": "Информация успешно собрана",
    "collected_at": collected_data["metadata"]["collection_time"],
    "data_quality": "real_only",
    "saved_to": DATA_FOLDER
}

except Exception as e:
    print(f"Ошибка сбора данных: {e}")
    return JSONResponse(
        status_code=500,
        content={"status": "error", "message": str(e)}
    )

# АДМИН-ПАНЕЛЬ
@app.get("/admin")
async def admin_panel():
    """Веб-панель администратора"""
    stats = await get_statistics()

    html = f"""
<!DOCTYPE html>
<html>
<head>
<title>Админ-панель - Жук Анастасия Лаб 3</title>
<meta charset="utf-8">
<style>
body {{ font-family: Arial, sans-serif; margin: 30px; background: #f5f5f5; }}
.header {{ background: linear-gradient(to right, #2c3e50, #4a6491);
    color: white; padding: 30px; border-radius: 10px; margin-bottom: 30px; }}
.stats-grid {{ display: grid; grid-template-columns: repeat(auto-fit, minmax(200px,
1fr));
    gap: 20px; margin: 30px 0; }}
.stat-card {{ background: white; padding: 25px; border-radius: 10px;
    box-shadow: 0 5px 15px rgba(0,0,0,0.1); text-align: center; }}
.stat-number {{ font-size: 42px; font-weight: bold; color: #2c3e50; margin: 10px 0;
}}
.stat-label {{ color: #666; font-size: 16px; }}
table {{ width: 100%; background: white; border-radius: 10px; overflow: hidden;
    box-shadow: 0 5px 15px rgba(0,0,0,0.1); border-collapse: collapse; }}
th {{ background: #2c3e50; color: white; padding: 15px; text-align: left; }}
td {{ padding: 12px 15px; border-bottom: 1px solid #eee; }}
tr:hover {{ background: #f9f9f9; }}
.mobile-badge {{ background: #e3f2fd; color: #1565c0; padding: 5px 10px;
    border-radius: 20px; font-size: 12px; }}
.desktop-badge {{ background: #f3e5f5; color: #7b1fa2; padding: 5px 10px;
    border-radius: 20px; font-size: 12px; }}
.tablet-badge {{ background: #e8f5e8; color: #2e7d32; padding: 5px 10px;

```

```

        border-radius: 20px; font-size: 12px; }}
    .filter-controls {{ margin: 20px 0; padding: 15px; background: white; border-radius:
10px; }}
    select, input {{ padding: 8px; margin-right: 10px; border-radius: 5px; border: 1px
solid #ddd; }}
    button {{ padding: 8px 15px; background: #2c3e50; color: white; border: none;
border-radius: 5px; cursor: pointer; }}
    button:hover {{ background: #4a6491; }}
    .section {{ margin: 30px 0; }}
    .chart-container {{ background: white; padding: 20px; border-radius: 10px; margin:
20px 0; }}
</style>
</head>
<body>
    <div class="header">
        <h1>Админ-панель сбора данных</h1>
        <p>Лабораторная работа №3 (Часть Б) | Студент: Жук Анастасия</p>
        <p>Данные сохраняются в: {DATA_FOLDER}</p>
    </div>

    <div class="stats-grid">
        <div class="stat-card">
            <div class="stat-number">{stats['total_clients']}</div>
            <div class="stat-label">Уникальных клиентов</div>
        </div>
        <div class="stat-card">
            <div class="stat-number">{stats['total_requests']}</div>
            <div class="stat-label">Всего запросов</div>
        </div>
        <div class="stat-card">
            <div class="stat-number">{stats['devices']['mobile']}</div>
            <div class="stat-label">Мобильные устройства</div>
        </div>
        <div class="stat-card">
            <div class="stat-number">{stats['devices']['desktop']}</div>
            <div class="stat-label">Компьютеры</div>
        </div>
        <div class="stat-card">
            <div class="stat-number">{stats['devices'].get('tablet', 0)}</div>
            <div class="stat-label">Планшеты</div>
        </div>
    </div>

    <div class="section">
        <h2>Список клиентов</h2>
        <div class="filter-controls">
            <select id="deviceFilter">
                <option value="all">Все устройства</option>
                <option value="desktop">Компьютеры</option>
                <option value="mobile">Мобильные</option>
            </select>
        </div>
    </div>

```

```

        <option value="tablet">Планшеты</option>
    </select>
    <input type="text" id="searchInput" placeholder="Поиск по IP...">
    <button onclick="filterTable()">Фильтровать</button>
    <button onclick="resetFilters()">Сбросить</button>
</div>
<table id="clientsTable">
    <tr>
        <th>IP адрес</th>
        <th>Устройство</th>
        <th>ОС / Платформа</th>
        <th>Браузер</th>
        <th>Разрешение</th>
        <th>Запросов</th>
        <th>Последний визит</th>
    </tr>

```

```

for client in stats["clients_list"]:
    if client["device"] == "mobile":
        badge = '<span class="mobile-badge">МОБИЛЬНЫЙ</span>'
    elif client["device"] == "tablet":
        badge = '<span class="tablet-badge">ПЛАШЕТ</span>'
    else:
        badge = '<span class="desktop-badge">КОМПЬЮТЕР</span>'

```

```

html += f"""
    <tr>
        <td><strong>{client['ip']}</strong></td>
        <td>{badge}</td>
        <td>{client['platform']}</td>
        <td>{client['browser']}</td>
        <td>{client['resolution']}</td>
        <td>{client['requests']}</td>
        <td>{client['last_seen']}</td>
    </tr>
"""

```

```

html += """
    </table>
</div>

```

```

<div class="section">
    <h2>Статистика</h2>
    <div class="chart-container">
        <h3>Распределение по операционным системам</h3>
    </div>

```

```

for platform, count in sorted(stats["platforms"].items(), key=lambda x: x[1],
reverse=True)[:5]:

```

```

percent = (count / stats['total_clients'] * 100) if stats['total_clients'] > 0 else 0
html += f'<p>{platform}: {count} клиентов ({percent:.1f}%)</p>'

html += """
    </div>

    <div class="chart-container">
        <h3>Распределение по браузерам</h3>
        """

for browser, count in sorted(stats["browsers"].items(), key=lambda x: x[1], reverse=True):
    percent = (count / stats['total_clients'] * 100) if stats['total_clients'] > 0 else 0
    html += f'<p>{browser}: {count} ({percent:.1f}%)</p>'

html += f'"""
    </div>
</div>

<div style="margin-top: 30px; padding: 20px; background: white; border-radius:
10px;">
    <h3>Информация о сервере</h3>
    <p>Сервер запущен: {datetime.now().strftime("%d.%m.%Y %H:%M")}</p>
    <p>Папка данных: <code>{os.path.abspath(DATA_FOLDER)}</code></p>
    <p>Всего файлов данных: {stats['total_files']}</p>
    <p><a href="/">Страница сбора данных</a></p>
</div>

<script>
function filterTable() {{
    const deviceFilter = document.getElementById('deviceFilter').value;
    const searchText = document.getElementById('searchInput').value.toLowerCase();
    const rows = document.querySelectorAll('#clientsTable tr');

    for (let i = 1; i < rows.length; i++) {{
        const row = rows[i];
        const deviceCell = row.cells[1].textContent;
        const ipCell = row.cells[0].textContent.toLowerCase();

        let deviceMatch = true;
        let searchMatch = true;

        if (deviceFilter !== 'all') {{
            if (deviceFilter === 'desktop' && !deviceCell.includes('КОМПЬЮТЕР'))
deviceMatch = false;
            if (deviceFilter === 'mobile' && !deviceCell.includes('МОБИЛЬНЫЙ'))
deviceMatch = false;
            if (deviceFilter === 'tablet' && !deviceCell.includes('ПЛИАНШЕТ'))
deviceMatch = false;
        }}
    }}

```

```

        if (searchText && !ipCell.includes(searchText)) {{
            searchMatch = false;
        }}

        row.style.display = (deviceMatch && searchMatch) ? "" : 'none';
    }}
}}

function resetFilters() {{
    document.getElementById('deviceFilter').value = 'all';
    document.getElementById('searchInput').value = "";
    const rows = document.querySelectorAll('#clientsTable tr');
    for (let i = 1; i < rows.length; i++) {{
        rows[i].style.display = "";
    }}
}}
</script>
</body>
</html>
"""

```

```

return HTMLResponse(html)

```

```

async def get_statistics():

```

```

    """Статистика собранных данных"""

```

```

    stats = {
        "total_clients": 0,
        "total_requests": 0,
        "total_files": 0,
        "devices": {"mobile": 0, "desktop": 0, "tablet": 0},
        "platforms": {},
        "resolutions": {},
        "browsers": {},
        "clients_list": []
    }

```

```

    if os.path.exists(DATA_FOLDER):

```

```

        for item in os.listdir(DATA_FOLDER):

```

```

            item_path = os.path.join(DATA_FOLDER, item)

```

```

            if os.path.isdir(item_path) and item.count("_") >= 3: # Папка IP

```

```

                stats["total_clients"] += 1

```

```

                files = [f for f in os.listdir(item_path) if f.endswith('.json')]

```

```

                stats["total_requests"] += len(files)

```

```

                stats["total_files"] += len(files)

```

```

            if files:

```

```

                latest_file = max(files)

```

```

                latest_path = os.path.join(item_path, latest_file)

```

```

            try:

```

```

with open(latest_path, 'r', encoding='utf-8') as f:
    data = json.load(f)

    client_data = data.get("client", {})
    system_data = data.get("system", {})

    device_type = client_data.get("device_type", "desktop")
    platform = system_data.get("detected_os", "Unknown")
    resolution = client_data.get("screen_resolution", "Unknown")
    browser = system_data.get("detected_browser", "Unknown")

    # Статистика
    if device_type in stats["devices"]:
        stats["devices"][device_type] += 1
    else:
        stats["devices"][device_type] = 1

    stats["platforms"][platform] = stats["platforms"].get(platform, 0) + 1
    stats["resolutions"][resolution] = stats["resolutions"].get(resolution, 0) + 1
    stats["browsers"][browser] = stats["browsers"].get(browser, 0) + 1

    # Список клиентов
    stats["clients_list"].append({
        "ip": item.replace("_", "."),
        "device": device_type,
        "platform": platform,
        "browser": browser,
        "resolution": resolution,
        "last_seen": data.get("metadata", {}).get("server_time"),
        "requests": len(files)
    })

except Exception as e:
    print(f"Ошибка чтения файла {latest_path}: {e}")
    continue

return stats

@app.get("/api/stats")
async def api_statistics():
    """API статистики в JSON формате"""
    stats = await get_statistics()
    return {
        "data_folder": DATA_FOLDER,
        "exists": os.path.exists(DATA_FOLDER),
        "statistics": stats
    }

# ЗАПУСК СЕРВЕРА
if __name__ == "__main__":

```

```

import uvicorn

print("\n" + "="*70)
print("СЕРВЕР ДЛЯ ЧАСТИ Б ЗАПУЩЕН")
print("="*70)
print(f"Студент: Жук Анастасия")
print(f"Папка данных (сетевой диск): {DATA_FOLDER}")
print(f"Статус: {'Доступна' if os.path.exists(DATA_FOLDER) else 'Проблемы'}")

# Получаем IP для доступа в сети
try:
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.connect(("8.8.8.8", 80))
    local_ip = s.getsockname()[0]
    s.close()
    print(f"Локальный IP: {local_ip}")
    print(f"Страница сбора: http://{local_ip}:8000")
    print(f"Админ-панель: http://{local_ip}:8000/admin")
except:
    print("Страница сбора: http://localhost:8000")
    print("Админ-панель: http://localhost:8000/admin")

print("="*70)
print("Для остановки: Ctrl+C")
print("="*70 + "\n")

uvicorn.run(app, host="0.0.0.0", port=8000)

```

Листинг 5 – Код анализатора analyzer.py

```

import json
import os
import subprocess
from datetime import datetime
from collections import Counter
import statistics

def find_data_folder():
    """Автоматически ищет папку с данными"""
    print("\n" + "="*60)
    print("ПОИСК ПАПКИ С ДАННЫМИ")
    print("="*60)

    possible_paths = [
        # 1. Сетевой диск Z:
        "Z:\\Zhuk_Lab3_Collected_Data",

        # 2. Прямой UNC путь
        r"\\LAPTOP-B87NA6T6\\ForAll\\Zhuk_Lab3_Collected_Data",

        # 3. Попробуем подключить сетевой диск

```



```

None, # Будет создан динамически

# 4. Локальная папка
"C:\\Zhuk_Lab3_Collected_Data",

# 5. Текущая директория
"./Zhuk_Lab3_Collected_Data"
]

# Пробуем подключить сетевой диск если нужно
network_path = r"\\LAPTOP-B87NA6T6\\ForAll"
print(f"1. Проверяем сетевой ресурс: {network_path}")

try:
    # Пробуем подключить диск
    result = subprocess.run(['net', 'use', 'Z:', network_path, '/persistent:yes'],
                            capture_output=True, text=True)

    if "successfully" in result.stdout.lower() or "успешно" in result.stdout.lower():
        print(" Сетевой диск подключен")
        possible_paths[0] = "Z:\\Zhuk_Lab3_Collected_Data"
    else:
        print(" Не удалось подключить сетевой диск")

    # Пробуем UNC напрямую
    if os.path.exists(network_path):
        print(" UNC путь доступен напрямую")
        possible_paths[1] = network_path + "\\Zhuk_Lab3_Collected_Data"
except:
    print(" Ошибка подключения сетевого диска")

# Проверяем все пути
for i, path in enumerate(possible_paths):
    if path and os.path.exists(path):
        print(f"\nНайдена папка с данными: {path}")
        print("="*60)
        return path

# Если ничего не нашли
print("\nПапка с данными не найдена!")
print("Проверенные пути:")
for path in possible_paths:
    if path:
        print(f" - {path}")

# Создаем папку локально для теста
local_path = "C:\\Zhuk_Lab3_Collected_Data"
print(f"\nСоздаю тестовую папку: {local_path}")
os.makedirs(local_path, exist_ok=True)

```

```

# Создаем тестовые данные
test_data = {
    "test": "Это тестовые данные, так как реальные не найдены",
    "created": datetime.now().isoformat()
}

test_file = os.path.join(local_path, "test_data.json")
with open(test_file, 'w', encoding='utf-8') as f:
    json.dump(test_data, f, indent=2)

print("Созданы тестовые данные")
print("="*60)
return local_path

def analyze_collected_data(data_folder=None):
    """Анализ данных и создание отчета"""

    if data_folder is None:
        data_folder = find_data_folder()

    print("\n" + "="*70)
    print("АНАЛИЗ СОБРАННЫХ ДАННЫХ - ЧАСТЬ Б")
    print("="*70)

    stats = {
        "data_folder": data_folder,
        "total_clients": 0,
        "total_files": 0,
        "devices": Counter(),
        "platforms": Counter(),
        "resolutions": Counter(),
        "browsers": Counter(),
        "timezones": Counter(),
        "cpu_cores": [],
        "gpu_vendors": Counter(),
        "clients": []
    }

    # Анализируем все папки
    for client_folder in os.listdir(data_folder):
        client_path = os.path.join(data_folder, client_folder)

        if os.path.isdir(client_path):
            stats["total_clients"] += 1
            files = [f for f in os.listdir(client_path) if f.endswith('.json')]
            stats["total_files"] += len(files)

        if files:
            # Берем последний файл для анализа
            latest_file = max(files)

```

```

latest_path = os.path.join(client_path, latest_file)

try:
    with open(latest_path, 'r', encoding='utf-8') as f:
        data = json.load(f)

    client_data = data.get("client", {})
    system_data = data.get("system", {})

    # Собираем статистику
    device_type = client_data.get("device_type", "desktop")
    platform = system_data.get("detected_os", "Unknown")
    resolution = client_data.get("screen_resolution", "Unknown")
    timezone = system_data.get("timezone", "Unknown")
    cpu_cores = system_data.get("hardware_concurrency")
    gpu_vendor = system_data.get("gpu_vendor", "Unknown")
    browser = system_data.get("detected_browser", "Unknown")

    stats["devices"][device_type] += 1
    stats["platforms"][platform] += 1
    stats["resolutions"][resolution] += 1
    stats["browsers"][browser] += 1
    stats["timezones"][timezone] += 1
    stats["gpu_vendors"][gpu_vendor] += 1

    if cpu_cores:
        stats["cpu_cores"].append(cpu_cores)

    # Добавляем клиента
    stats["clients"].append({
        "ip": client_folder.replace("_", "."),
        "device": device_type,
        "platform": platform,
        "browser": browser,
        "resolution": resolution,
        "timezone": timezone,
        "cpu_cores": cpu_cores,
        "gpu": gpu_vendor,
        "last_seen": data.get("metadata", {}).get("server_time"),
        "files": len(files)
    })

except Exception as e:
    print(f'⚠ Ошибка анализа файла {latest_path}: {e}')

# Вывод статистики
print(f'\nОБЩАЯ СТАТИСТИКА:')
print(f' Папка данных: {stats["data_folder"]}')
print(f' Уникальных клиентов: {stats["total_clients"]}')
print(f' Всего записей: {stats["total_files"]}')

```

```

print(f"\nУСТРОЙСТВА:")
for device, count in stats["devices"].most_common():
    percent = (count / stats["total_clients"] * 100) if stats["total_clients"] > 0 else 0
    print(f"  {device}: {count} ({percent:.1f}%)")

if stats["cpu_cores"]:
    print(f"\nПРОЦЕССОРЫ (ядра):")
    print(f"  Среднее количество: {statistics.mean(stats['cpu_cores']):.1f}")
    print(f"  Минимальное: {min(stats['cpu_cores'])}")
    print(f"  Максимальное: {max(stats['cpu_cores'])}")

print(f"\nОПЕРАЦИОННЫЕ СИСТЕМЫ:")
for platform, count in stats["platforms"].most_common(5):
    percent = (count / stats["total_clients"] * 100) if stats["total_clients"] > 0 else 0
    print(f"  {platform}: {count} ({percent:.1f}%)")

print(f"\nБРАУЗЕРЫ:")
for browser, count in stats["browsers"].most_common():
    percent = (count / stats["total_clients"] * 100) if stats["total_clients"] > 0 else 0
    print(f"  {browser}: {count} ({percent:.1f}%)")

print(f"\nРАЗРЕШЕНИЯ ЭКРАНА:")
for res, count in stats["resolutions"].most_common(5):
    percent = (count / stats["total_clients"] * 100) if stats["total_clients"] > 0 else 0
    print(f"  {res}: {count} ({percent:.1f}%)")

# Создаем отчет
report_file =
f"lab3_part_b_report_{datetime.now().strftime('%Y%m%d_%H%M%S')}.txt"
with open(report_file, 'w', encoding='utf-8') as f:
    f.write("=" * 80 + "\n")
    f.write("ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3 (ЧАСТЬ Б)\n")
    f.write("СТУДЕНТ: ЖУК АНАСТАСИЯ\n")
    f.write(f"ДАТА АНАЛИЗА: {datetime.now().strftime('%d.%m.%Y %H:%M:%S')}\n")
    f.write(f"ПАПКА ДАННЫХ: {stats['data_folder']}\n")
    f.write("=" * 80 + "\n\n")

    f.write(f"ОБЩАЯ СТАТИСТИКА\n")
    f.write(f"  Уникальных клиентов: {stats['total_clients']}\n")
    f.write(f"  Всего записей: {stats['total_files']}\n\n")

    f.write(f"РАСПРЕДЕЛЕНИЕ ПО УСТРОЙСТВАМ\n")
    for device, count in stats["devices"].most_common():
        percent = (count / stats["total_clients"] * 100) if stats["total_clients"] > 0 else 0
        f.write(f"  {device.upper():<20}: {count} ({percent:.1f}%)")

    if stats["cpu_cores"]:
        f.write(f"\nХАРАКТЕРИСТИКИ ПРОЦЕССОРОВ\n")
        f.write(f"  Среднее количество ядер: {statistics.mean(stats['cpu_cores']):.1f}\n")

```

```

f.write(f"  Всего ядер у всех клиентов: {sum(stats['cpu_cores'])}\n")

f.write(f"\nОПЕРАЦИОННЫЕ СИСТЕМЫ\n")
for platform, count in stats["platforms"].most_common(10):
    percent = (count / stats["total_clients"] * 100) if stats["total_clients"] > 0 else 0
    f.write(f"  {platform}: {count} ({percent:.1f}%) \n")

f.write(f"\nБРАУЗЕРЫ\n")
for browser, count in stats["browsers"].most_common():
    percent = (count / stats["total_clients"] * 100) if stats["total_clients"] > 0 else 0
    f.write(f"  {browser}: {count} ({percent:.1f}%) \n")

f.write(f"\nДЕТАЛЬНЫЙ СПИСОК КЛИЕНТОВ\n")
f.write(f"=" * 100 + "\n")
for client in stats["clients"]:
    f.write(f"IP: {client['ip']}\n")
    f.write(f"  Устройство: {client['device']}\n")
    f.write(f"  ОС: {client['platform']}\n")
    f.write(f"  Браузер: {client['browser']}\n")
    f.write(f"  Разрешение: {client['resolution']}\n")
    f.write(f"  Часовой пояс: {client['timezone']}\n")
    f.write(f"  Ядер CPU: {client['cpu_cores']} or 'Неизвестно'\n")
    f.write(f"  GPU: {client['gpu']}\n")
    f.write(f"  Последний визит: {client['last_seen']}\n")
    f.write(f"  Запросов: {client['files']}\n")
    f.write(f"-" * 100 + "\n")

print(f"\nОтчет сохранен: {report_file}")
print(f"=" * 70)

return stats

if __name__ == "__main__":
    analyze_collected_data()

```

Примеры работы программы

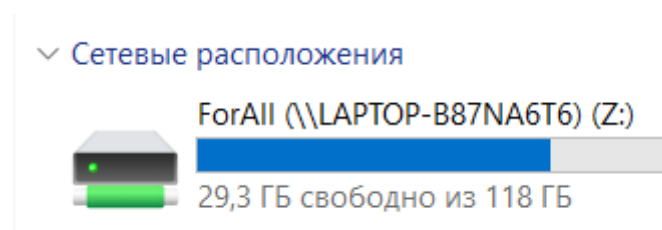


Рисунок 10 – Сетевой диск на другом ноутбуке, доступный моему ноутбуку

```
Microsoft Windows [Version 10.0.26100.7462]
(c) Microsoft Corporation. All rights reserved.
Active code page: 65001

C:\Users\Анастасия>cd C:\py.project\first\timp_lab3_b

C:\py.project\first\timp_lab3_b>python server.py

=====
НАСТРОЙКА СЕТЕВОГО ДИСКА
=====
1. Проверяем доступность: \\LAPTOP-B87NA6T6\ForAll
   Компьютер в сети
2. Подключаем сетевой диск Z: → \\LAPTOP-B87NA6T6\ForAll
   Сетевой диск подключен: Z:

Папка для данных: Z:\Zhuk_Lab3_Collected_Data
=====

=====
СЕРВЕР ДЛЯ ЧАСТИ Б ЗАПУЩЕН
=====
Студент: Жук Анастасия
Папка данных (сетевой диск): Z:\Zhuk_Lab3_Collected_Data
Статус: Доступна
Локальный IP: 192.168.31.252
Страница сбора: http://192.168.31.252:8000
Админ-панель: http://192.168.31.252:8000/admin
=====
Для остановки: Ctrl+C
=====

INFO: Started server process [22252]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Рисунок 11 – Запуск сервера

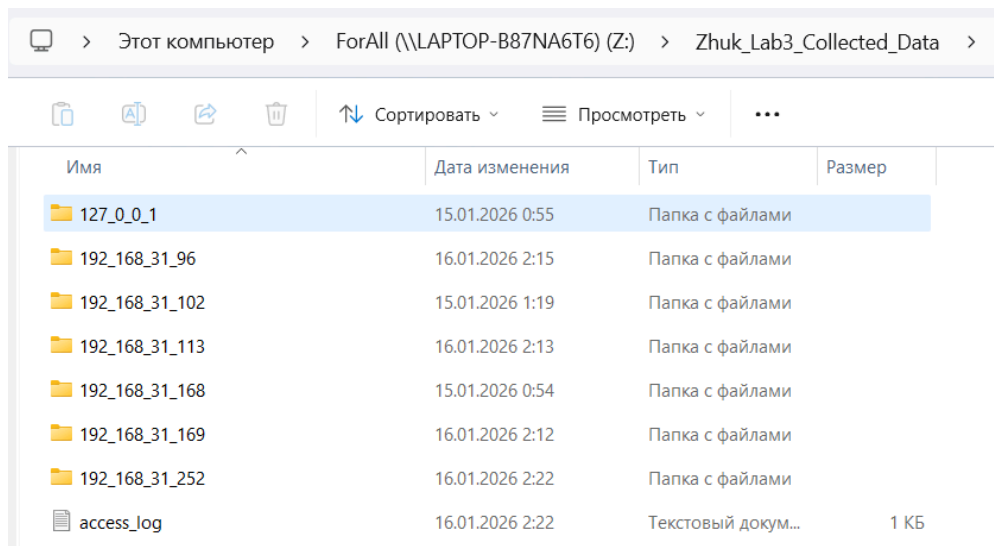


Рисунок 12 – Содержимое сетевой папки на сетевом диске

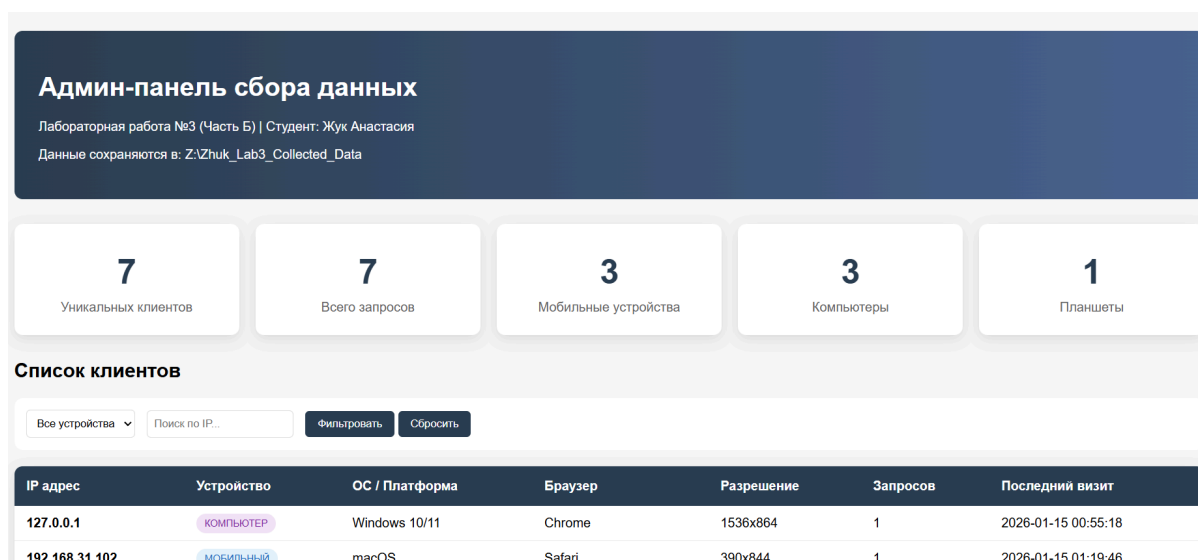


Рисунок 13 – Панель администратора, на которой можно увидеть всех пользователей, чью информацию удалось собрать

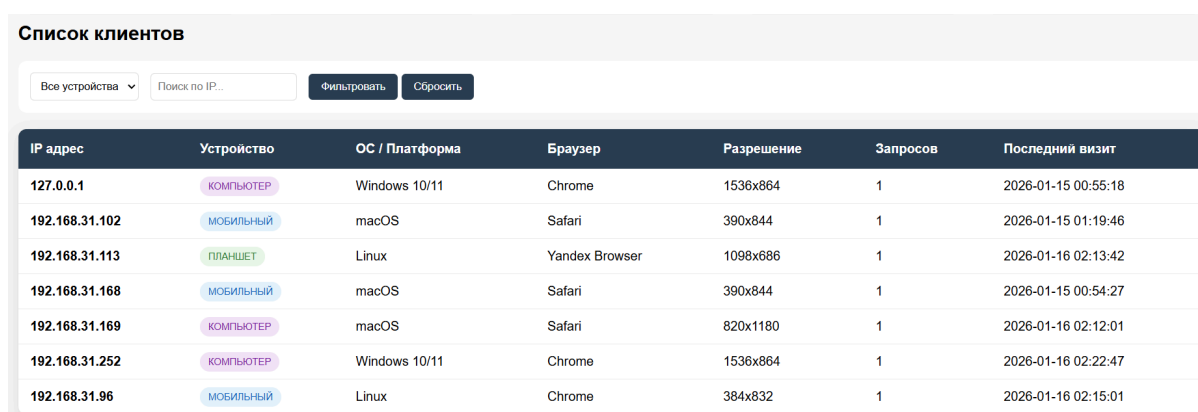


Рисунок 14 – Список клиентов на панели администратора

Список клиентов

Компьютеры

Все устройства

Компьютеры

Мобильные

Планшеты

Поиск по IP...

Фильтровать

Сбросить

	Устройство	ОС / Платформа	Браузер	Разрешение	Запросов	Последний визит
127.0.0.1	КОМПЬЮТЕР	Windows 10/11	Chrome	1536x864	1	2026-01-15 00:55:18
192.168.31.169	КОМПЬЮТЕР	macOS	Safari	820x1180	1	2026-01-16 02:12:01
192.168.31.252	КОМПЬЮТЕР	Windows 10/11	Chrome	1536x864	1	2026-01-16 02:22:47

Рисунок 15 – Сортировка клиентов на панели администратора

Статистика

Распределение по операционным системам

macOS: 3 клиентов (42.9%)

Windows 10/11: 2 клиентов (28.6%)

Linux: 2 клиентов (28.6%)

Распределение по браузерам

Chrome: 3 (42.9%)

Safari: 3 (42.9%)

Yandex Browser: 1 (14.3%)

Рисунок 16 – Статистика на панели администратора

Информация о сервере

Сервер запущен: 16.01.2026 02:25

Папка данных: Z:\Zhuk_Lab3_Collected_Data

Всего файлов данных: 7

[Страница сбора данных](#)

Рисунок 17 – Информация о сервере на панели администратора

Технологии будущего

Актуальные новости, аналитика и обзоры IT-индустрии

Искусственный интеллект в повседневной жизни

С каждым годом искусственный интеллект становится все более интегрированным в нашу повседневную жизнь. От умных помощников в смартфонах до сложных систем управления городской инфраструктурой - ИИ меняет то, как мы живем и работаем.

"Мы находимся на пороге новой технологической революции, где ИИ будет не просто инструментом, а партнером в решении сложных задач" - говорит ведущий эксперт в области машинного обучения.

В последние годы произошел значительный прорыв в области обработки естественного языка. Модели, такие как GPT-4, способны генерировать тексты, неотличимые от человеческих, переводить языки с высокой точностью и даже писать программный код.

Последние новости

Новый прорыв в солнечных батареях

Ученые разработали солнечные элементы с КПД 47%, что приближает нас к эре дешевой возобновляемой энергии.

Кибербезопасность в 2024

Эксперты предупреждают о новых типах атак на системы искусственного интеллекта.

Обновление ОС Windows

Microsoft анонсировала крупное

Рисунок 18 – Страница сбора данных

76

АНАЛИЗ СОБРАННЫХ ДАННЫХ – ЧАСТЬ Б

ОБЩАЯ СТАТИСТИКА:

Папка данных: Z:\Zhuk_Lab3_Collected_Data

Уникальных клиентов: 7

Всего записей: 7

УСТРОЙСТВА:

desktop: 3 (42.9%)

mobile: 3 (42.9%)

tablet: 1 (14.3%)

ПРОЦЕССОРЫ (ядра):

Среднее количество: 8.0

Минимальное: 4

Максимальное: 12

ОПЕРАЦИОННЫЕ СИСТЕМЫ:

macOS: 3 (42.9%)

Windows 10/11: 2 (28.6%)

Linux: 2 (28.6%)

БРАУЗЕРЫ:

Chrome: 3 (42.9%)

Safari: 3 (42.9%)

Yandex Browser: 1 (14.3%)

РАЗРЕШЕНИЯ ЭКРАНА:

1536x864: 2 (28.6%)

390x844: 2 (28.6%)

1098x686: 1 (14.3%)

820x1180: 1 (14.3%)

384x832: 1 (14.3%)

Отчет сохранен: lab3_part_b_report_20260116_022948.txt

Рисунок 19 – Запуск analyzer.py. Создание отчета об устройствах

```
mobile_20260116_021501_969.json X
Z: > Zhuk_Lab3_Collected_Data > 192.168.31.96 > {} mobile_20260116_021501_969.json > ...
1  {}
2  "metadata": {
3    "collection_id": "e49dd26f-fe7a-4209-a0fa-9df20a5cf10e",
4    "collection_time": "2026-01-16T02:15:01.929039",
5    "server_time": "2026-01-16 02:15:01",
6    "server_hostname": "DESKTOP-5SJE5GJ",
7    "server_os": "Windows 10"
8  },
9  "client": {
10   "ip_address": "192.168.31.96",
11   "user_agent": "Mozilla/5.0 (Linux; Android 15; K) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.7499.192 Mobile Safari/537.36",
12   "device_type": "mobile",
13   "platform": "Linux aarch64",
14   "screen_resolution": "384x832",
15   "referrer": "direct",
16   "public_ip": "91.196.254.77",
17   "local_ips": [
18     "192.168.31.96"
19   ]
20 }
```

Рисунок 20 – Содержимое JSON файлов об устройстве (1)

```
21 "system": {
22   "languages": [
23     "ru",
24     "ru-RU",
25     "en-US"
26   ],
27   "timezone": "Europe/Moscow",
28   "hardware_concurrency": 8,
29   "cookie_enabled": true,
30   "connection_type": "4g",
31   "downlink": 10.0,
32   "rtt": 0,
33   "detected_os": "Linux",
34   "detected_browser": "Chrome",
35   "color_depth": 24,
36   "pixel_ratio": 2.8125,
37   "orientation": "portrait-primary",
38   "max_touch_points": 5,
39   "gpu_vendor": "ARM",
40   "gpu_renderer": "Mali-G57 MC2"
41 }
42 }
```

Рисунок 21 – Содержимое JSON файлов об устройстве (2)

ЗАКЛЮЧЕНИЕ

Выполнение лабораторной работы позволило создать две взаимодополняющие системы, демонстрирующие различные аспекты работы с информацией в современных компьютерных системах. В первой части работы была разработана система защиты локальной системной информации, которая использует современные криптографические методы для обеспечения конфиденциальности и целостности данных. Этот подход показал, как можно эффективно защищать информацию, используя электронные подписи и шифрование, при этом интегрируя защитные механизмы непосредственно в операционную систему через реестр и ассоциации файлов.

Вторая часть работы представила принципиально иной подход – удалённый сбор технической информации через веб-интерфейсы. Разработанная система демонстрирует, как современные браузерные технологии позволяют собирать подробные данные об устройствах пользователей без их явного согласия или установки дополнительного программного обеспечения. Этот аспект работы имеет особую актуальность в контексте понимания возможностей и рисков, связанных с современными веб-технологиями.

Обе системы, несмотря на их внешние различия, объединены общей идеей – управлением информацией в цифровой среде. Первая система защищает информацию от несанкционированного доступа, вторая – собирает информацию для анализа и статистики. Вместе они создают целостную картину того, как информация может и защищаться, и собираться в современных компьютерных системах.

Практическая реализация обеих частей работы подтвердила важность не только теоретических знаний в области криптографии и веб-технологий, но и умения применять эти знания для создания работающих, устойчивых систем. Разработанные решения учитывают

различные сценарии использования, обрабатывают исключительные ситуации и предоставляют удобные интерфейсы для пользователей.

В результате выполнения работы были не только созданы функциональные программные продукты, но и получен опыт в области проектирования сложных систем, работы с сетевыми протоколами, криптографическими алгоритмами и современными веб-технологиями.